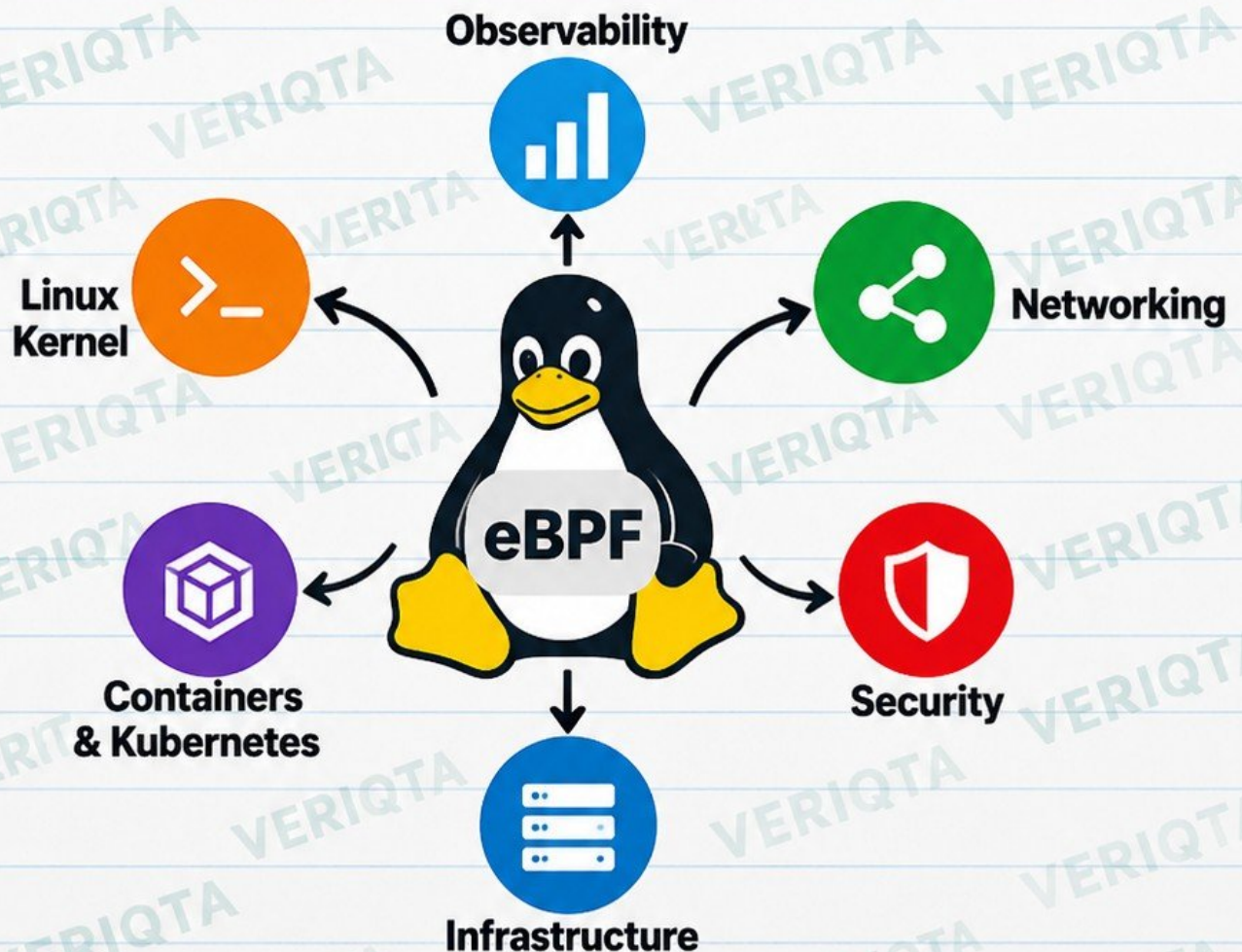


# eBPF

for DevOps & Infrastructure Engineers

From Fundamentals to Production



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



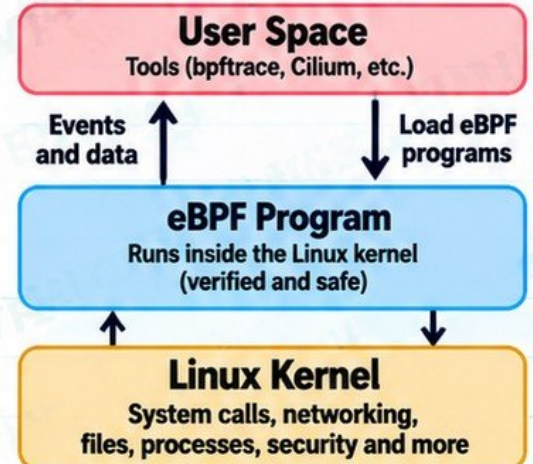
@veriqta\_academy

# 01 What Is eBPF?

## What Is eBPF?

- eBPF (extended Berkeley Packet Filter) is a technology that allows small, safe programs to run inside the Linux kernel.
- It can observe, filter, and process events such as system calls, network traffic, file operations and more.
- eBPF programs run in the kernel but are controlled by user-space tools.

## How eBPF Fits in the Linux System



## Why Does eBPF Exist?

- Traditional tools could not easily and safely observe what is happening inside the kernel.
- eBPF was created to give engineers a safe, flexible and high-performance way to collect deep system visibility without changing the kernel source code.
- It allows new tools to be built quickly for observability, networking, security and performance analysis.

## What Problem Does It Solve?

- It helps you understand behaviour inside the kernel without adding heavy overhead.
- It provides real-time visibility into processes, network traffic, file activity and system calls.
- It enables modern tools for monitoring, troubleshooting, container networking and runtime security.
- It reduces the need for slow and complex kernel modules.

## Where Do Engineers Use It?

- Troubleshoot performance issues
- Analyse system calls and process behaviour
- Monitor network traffic and latency
- Improve Kubernetes and container networking
- Build security and threat detection tools
- Understand how applications interact with the kernel

## Real World Example

- A web application is slow.
- Using eBPF, you can observe which processes are making slow disk or network calls.
- You can see the exact system calls, latency and the process responsible.
- This helps you find the real cause faster than traditional tools.



### Junior Engineer Tip

eBPF is not just for developers. As a DevOps or infrastructure engineer, you use eBPF-based tools to gain deep visibility into your systems, containers and Kubernetes clusters.



### Important to Know

- eBPF programs are verified by the kernel for safety.
- They cannot crash the kernel.
- They run with very low overhead.
- You do not need to modify the kernel to use eBPF.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

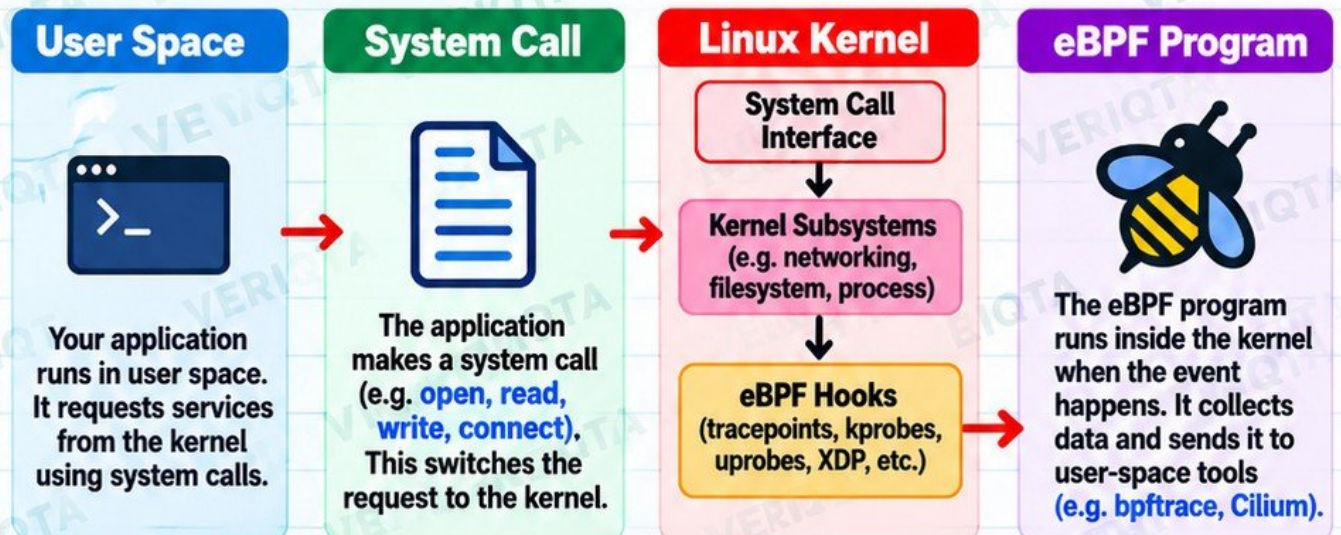
## 02 Where eBPF Runs

### Overview

- eBPF programs run inside the Linux kernel.
- They attach to specific kernel events such as system calls, network packets or file operations.
- User-space tools load, configure and read the data collected by eBPF programs.

### Why User Space Cannot See Everything

- Applications run in user space and only see their own process information.
- Many important events happen inside the kernel and are not visible to normal user-space tools.
- eBPF gives a safe way to observe these kernel events without changing the kernel code.



### Key Components

- **User space:** Your applications and eBPF tools (e.g. bpftrace, Cilium, etc.).
- **System call:** The interface used to request a service from the kernel.
- **Linux kernel:** Handles system calls, networking, files, processes and more.
- **eBPF program:** Runs inside the kernel at specific event points and collects data.
- **eBPF map:** Used to store and share data between the kernel and user space.

### Examples of Kernel Events

- System calls ([open](#), [read](#), [write](#), [connect](#))
- Network packets (incoming and outgoing)
- Process creation and execution
- File operations (open, close, read, write)
- Scheduler events (when processes are scheduled)
- Security events (e.g. permission checks)
- Container and Kubernetes events

### Important to Know

- eBPF programs are verified by the kernel for safety.
- They run in a restricted environment.
- They cannot crash the kernel.
- They only run at allowed kernel event points.

### Junior Engineer Tip

Think of eBPF as a small program that sits inside the kernel and listens for specific events. It collects useful data and sends it to user-space tools so you can understand what is happening in your system.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

## 03 Why Observe the Kernel?

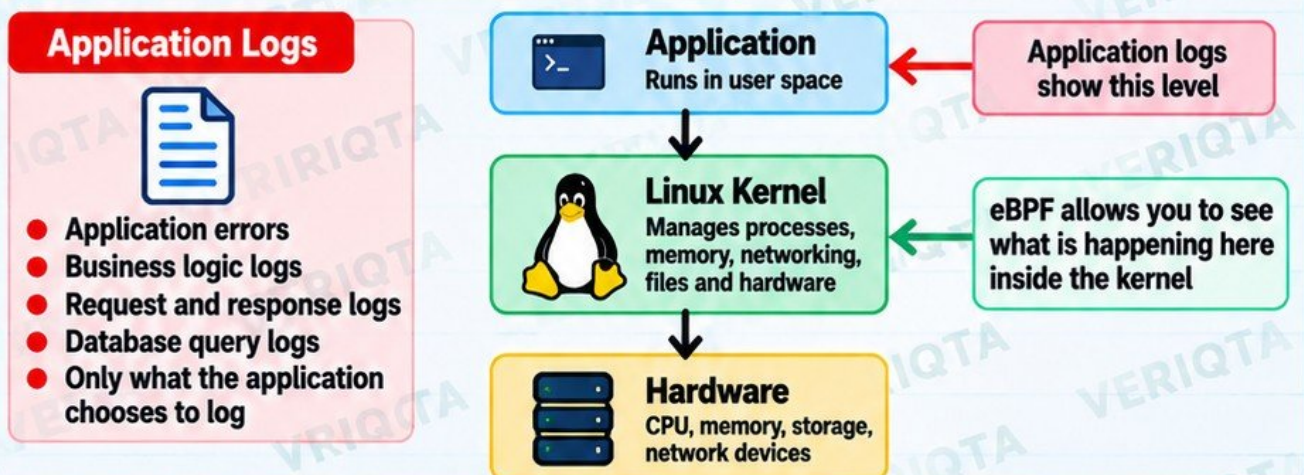
### The Problem with Application Logs

- Application logs only show what the application chooses to log.
- They do not show everything happening in the operating system or the kernel.
- Important issues such as network latency, disk I/O delays, system call errors or resource contention may not appear in application logs.
- You can have a healthy application log but still have performance or infrastructure problems.

### Why the Kernel Matters

- The kernel manages processes, memory, networking, files and hardware.
- Every application depends on the kernel for these operations.
- Observing the kernel gives you visibility into what is really happening in the system.
- It helps you find the true root cause instead of guessing.
- This is why eBPF is valuable. It allows you to safely observe kernel activity in real time.

### What You Can See with Logs vs eBPF



### Real World Example

- 1 Users report the application is slow.
- 2 Application logs show no errors.
- 3 Using eBPF, you observe high disk I/O wait in the kernel.
- 4 You find a process performing heavy file reads causing the slowdown.
- 5 You fix the issue and the application responds normally.

### Other Problems You Can Find

- High CPU usage and what is causing it
- Memory pressure and page faults
- Slow disk I/O and which process is responsible
- Network latency, dropped packets or connection issues
- Processes making unexpected system calls
- Container and Kubernetes node level issues
- File access and permission related problems
- Security related events and suspicious behaviour



### Junior Engineer Tip

When troubleshooting, do not rely only on application logs. Always consider what is happening in the kernel.



### Remember This

Application logs tell you what the application sees. The kernel tells you what is actually happening.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

## 04 Events Inside Linux

### What Are Kernel Events?

- The Linux kernel handles many activities on your system.
- These activities generate events that can be observed with eBPF.
- Events include system calls, process activity, network packets, file operations, scheduling and more.
- These events show what is really happening inside your system.

### Why These Events Matter

- They help you understand system behaviour at a deeper level.
- You can find the real cause of performance problems.
- You can see what processes are doing, which files they access and how they use the network.
- This visibility is useful for troubleshooting, monitoring, security and performance analysis.

### Common Types of Kernel Events

#### Processes



- Process creation (exec)
- Process exit
- Process state changes
- CPU usage

#### System Calls



- open, read, write
- connect, accept
- stat, chmod
- fork, execve
- and many more

#### Network Packets



- Incoming packets
- Outgoing packets
- Connection events
- Dropped packets
- Latency information

#### File Operations



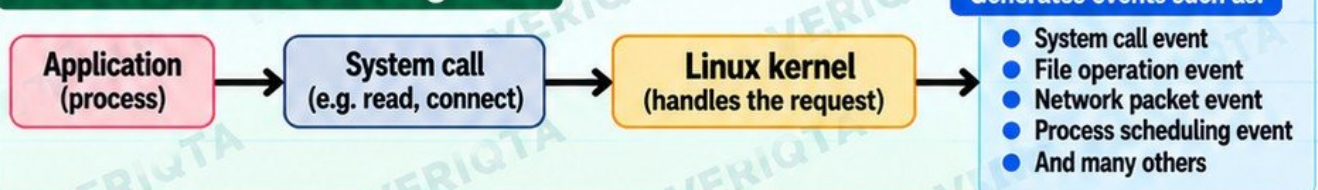
- File open
- File read/write
- File close
- File delete
- File permission changes

#### Scheduling



- Process scheduling
- CPU time usage
- Context switches
- Run queue activity
- Scheduling delays

### How These Events Fit Together



### Real World Example

- 1 A web application is slow.
- 2 You observe network events to check for latency.
- 3 You check system calls to see if the application is waiting for a file or a network connection.
- 4 You look at process scheduling to see if the process is getting enough CPU time.
- 5 You combine these events to find the real cause.

### Other Observable Events

- Memory allocation and page faults
- Socket operations
- Disk I/O requests and completion
- Timers and scheduler events
- Container and namespace events
- Security related events (e.g. permission checks)
- Cgroup events (resource limits)

### Junior Engineer Tip

Think of kernel events as a timeline of everything happening inside your system. eBPF helps you see this timeline in real time.

### Remember This

Many problems are not visible in application logs. Observing kernel events gives you the missing information you need to troubleshoot effectively.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

## 05 Traditional Tools vs eBPF

### What Are Traditional Tools?

- Tools such as top, strace, tcpdump and others have been used for many years to understand what is happening on Linux systems.
- They help you investigate performance issues, process behaviour, network traffic and more.
- They work well but each tool only shows a part of the picture.

### What Makes eBPF Different?

- eBPF runs inside the Linux kernel and can observe many types of events.
- It can collect data across processes, network, files, scheduling and more.
- It gives you deep visibility with low overhead and without modifying kernel code.
- It complements traditional tools and provides a more complete view of what is happening.

### Common Tools and What They Do

#### top



- Shows real-time CPU and memory usage
- Helps you find which processes are using resources
- Good for quick high-level overview

#### strace



- Traces system calls made by a process
- Shows what the application is asking the kernel to do
- Useful for debugging specific application issues

#### tcpdump



- Captures and analyses network packets
- Helps you see network traffic, protocols and connection details
- Useful for network troubleshooting

#### eBPF



- Runs inside the kernel at specific event points
- Can observe processes, system calls, network, files, scheduling and more
- Provides deep visibility with low overhead
- Used by modern observability, networking and security tools

### Side by Side Comparison

| Feature       | top                                       | strace                          | tcpdump                           | eBPF                                           |
|---------------|-------------------------------------------|---------------------------------|-----------------------------------|------------------------------------------------|
| Main focus    | CPU and memory                            | System calls                    | Network packets                   | Multiple kernel events                         |
| Where it runs | User space                                | User space                      | User space                        | Kernel (eBPF program)                          |
| Data source   | /proc and kernel metrics                  | System call trace               | Network packets                   | Kernel event hooks                             |
| Scope         | High-level process view                   | Single process                  | Network traffic                   | Processes, network, files, scheduling and more |
| Overhead      | Low                                       | Higher (per process)            | Moderate to high (packet capture) | Low (optimized)                                |
| Best used for | Quick resource check                      | Debugging application behaviour | Network troubleshooting           | Deep visibility and production observability   |
| Limitations   | Does not show why the process is doing it | Only sees one process           | Only network traffic              | Requires kernel support and eBPF tools         |



#### Key Takeaway

Traditional tools and eBPF are not competitors. They complement each other. Use the right tool for the problem, and use eBPF when you need deeper and broader visibility.



#### Real World Example

- top shows a process using high CPU.
- strace shows the system calls it is making.
- tcpdump shows the network traffic it sends.
- eBPF gives a complete view across processes, network, files and scheduling to find the real cause.



#### Remember This

If you only rely on traditional tools, you may miss important events inside the kernel. eBPF helps you see the bigger picture.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

## 06 The eBPF Execution Model

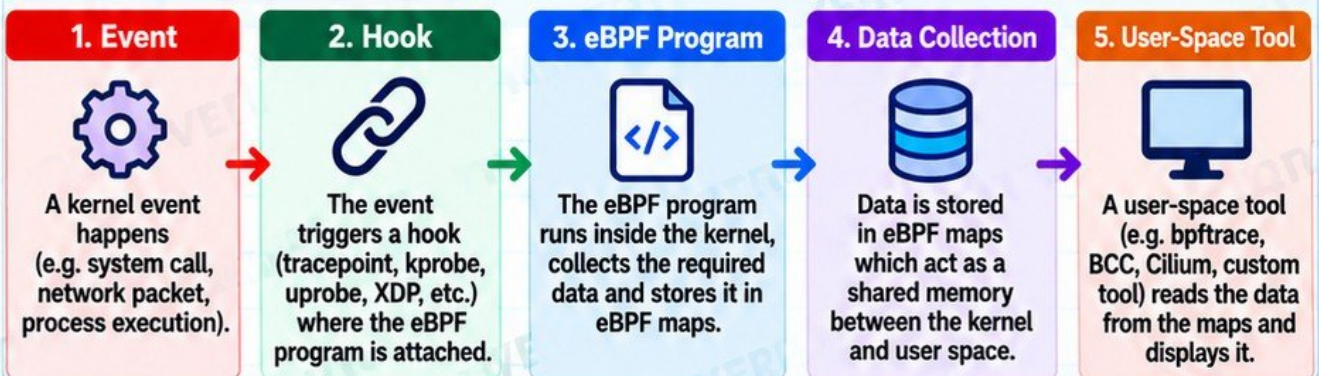
### What Is the eBPF Execution Model?

- The eBPF execution model defines how an event inside the Linux kernel is captured, processed and sent to user-space tools.
- A small eBPF program is attached to a specific kernel event (hook).
- When the event occurs, the eBPF program runs, collects useful data and sends it to user space.
- User-space tools then read, process and display the data.

### Key Points to Understand

- eBPF programs run inside the Linux kernel in a safe and controlled environment.
- They are triggered by specific events such as system calls, process activity, network packets and more.
- The program collects relevant data and stores it in eBPF maps.
- User-space tools read the data from these maps and present it in a human-friendly way.
- This allows real-time visibility without modifying the kernel or adding heavy overhead.

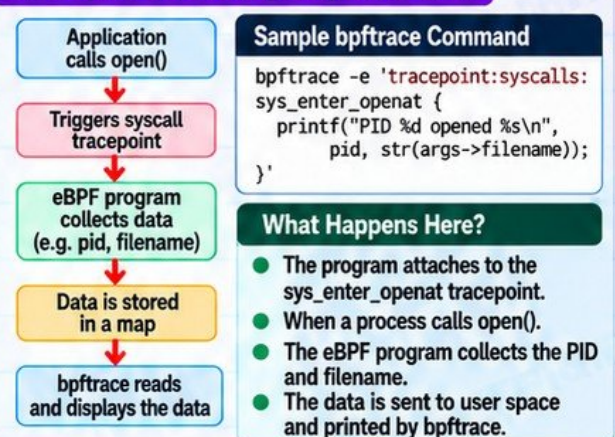
### The eBPF Execution Flow



### Common eBPF Hooks

| Hook Type  | What It Triggers On                           | Common Use Cases                              |
|------------|-----------------------------------------------|-----------------------------------------------|
| Tracepoint | Predefined kernel events                      | System calls, scheduling, I/O, network events |
| Kprobe     | Function entry                                | Trace kernel functions                        |
| Uprobe     | User-space function entry                     | Trace application functions                   |
| XDP        | Network packet arrival (before network stack) | High-performance network processing           |
| LSM        | Security events                               | Monitor and enforce security                  |
| Cgroup     | Process and cgroup events                     | Container and process monitoring              |

### Example: Observing a System Call



### Junior Engineer Tip

Think of the eBPF execution model as a pipeline. A kernel event triggers the eBPF program, the program collects useful data, and user-space tools present it in a readable format.

### Important to Know

- eBPF programs are verified by the kernel for safety.
- They can only run at allowed kernel event points.
- The data collected depends on the hook and program logic.
- User-space tools do not run inside the kernel. They only read the data.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 07 eBPF Hooks

## What Are eBPF Hooks?

- eBPF hooks are points inside the kernel or user space where an eBPF program can be attached and run.
- They are triggered by specific events such as system calls, function execution, network packet processing and more.
- Each hook type gives visibility into different parts of the system.
- Choosing the right hook depends on what you want to observe or achieve.

## Why eBPF Hooks Are Important?

- They allow you to observe real events as they happen inside the kernel and user space.
- They provide deep visibility into system behaviour without modifying the kernel.
- They can be used for observability, troubleshooting, security and networking.
- Different hooks give access to different layers such as application, kernel, network and storage.

## Common eBPF Hook Types

### Tracepoints



- Predefined instrumentation points in the kernel.
- Stable and safe to use.
- Provide structured event data.
- Good for general observability.

### kprobes



- Attach to almost any kernel function.
- Useful when no tracepoint is available.
- More flexible but less stable across kernel versions.
- Good for advanced troubleshooting.

### uprobes



- Attach to user space functions in an application.
- Useful for observing application behaviour without modifying the application.
- Commonly used to trace libraries such as libc or specific application functions.

### XDP



- Runs at the earliest point in the network stack (before the kernel processes the packet).
- Used for high performance networking, filtering and security.
- Very fast and runs in a restricted environment.

### Other Hooks



- LSM (Linux Security Module) hooks for security events.
- cgroup hooks for process and container events.
- perf events for performance analysis.
- raw tracepoints and others for specialized use cases.

## Where Each Hook Runs

User Space  
(Application)

→ uprobes  
(user space functions)

System Calls

→ tracepoints  
(syscall events)

Linux Kernel



→ kprobes  
(kernel functions)

→ LSM / cgroup hooks  
(security and process events)

Network Stack

→ XDP  
(network packets before kernel processing)

## Comparison of eBPF Hook Types

| Hook Type  | Where It Runs               | Main Use Case                                    | Pros                          | Limitations                                    |
|------------|-----------------------------|--------------------------------------------------|-------------------------------|------------------------------------------------|
| Tracepoint | Kernel (predefined)         | System events (syscalls, scheduling, I/O, etc.)  | Stable, structured data       | Limited to available events                    |
| kprobe     | Kernel (function entry)     | Any kernel function                              | Flexible, powerful            | May change across kernel versions              |
| uprobe     | User space (application)    | Application functions and libraries              | Observe application behaviour | Requires knowing the function name             |
| XDP        | Network stack (early layer) | High performance networking, filtering, security | Very fast, early visibility   | Limited functionality and stricter constraints |
| LSM        | Kernel (security hooks)     | Security and audit events                        | Good for security use cases   | Only security related events                   |
| cgroup     | Kernel (cgroup layer)       | Process and container events                     | Useful for containers         | Limited to cgroup events                       |



### Junior Engineer Tip

Start with tracepoints because they are stable and easy to use. Move to kprobes and uprobes when you need deeper visibility. Use XDP for network-related use cases at high speed.



### Remember This

- Different hooks are used for different problems.
- There is no single hook that covers everything.
- Always choose the right hook based on what you want to observe.
- Some hooks can be more sensitive to kernel version changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

## 08 The eBPF Verifier

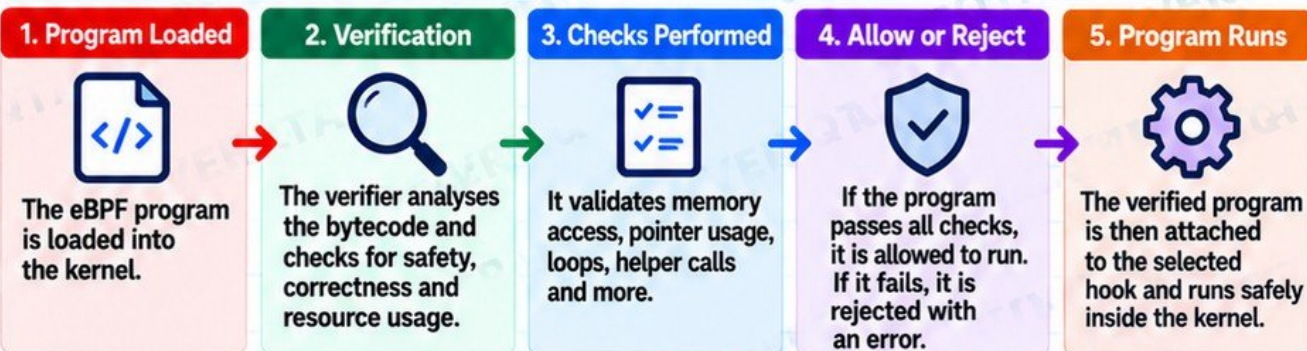
### What Is the eBPF Verifier?

- The eBPF verifier is a security and safety mechanism inside the Linux kernel.
- It checks every eBPF program before allowing it to run.
- The verifier analyses the program's instructions to make sure it is safe, correct and will not harm the kernel.
- Only programs that pass all checks are allowed to be loaded and executed.

### Why the Kernel Checks eBPF Programs?

- eBPF programs run inside the kernel with high privileges.
- A bug in a program can crash the kernel, access invalid memory or cause unstable behaviour.
- The verifier prevents unsafe code from running.
- It protects the kernel, your system and other applications.
- It allows eBPF to be safe and widely used in production environments.

### How the eBPF Verifier Works



### What the Verifier Checks

- **Memory Safety** – prevents invalid or unsafe memory access.
- **Pointer Validity** – ensures pointers are used correctly.
- **Bounds Checking** – verifies array and map access is within limits.
- **No Infinite Loops** – ensures loops will always terminate.
- **Helper Function Safety** – checks that only allowed helper functions are used correctly.
- **Resource Limits** – enforces limits on stack size, instructions and complexity.
- **Program Behaviour** – performs static analysis to predict all possible paths.

### Example: Verifier Rejects an Unsafe Program

```

1 SEC("tracepoint/syscalls/sys_enter_openat")
2 int unsafe_example(struct trace_event_raw_sys_enter *ctx)
3 {
4     int *ptr = (int *)0x12345678; // invalid memory access
5     bpf_probe_read_kernel(ptr, sizeof(int), 0x12345678);
6     return 0;
7 }
  
```

#### Verifier Error (example)

```

error: invalid memory access 'ptr'
R0 invalid mem access, reg type=inv
processed 6 insns (limit 1000000) max_states_per_insn 0 total_states 0
  
```

### Example: A Valid Program

```

1 SEC("tracepoint/syscalls/sys_enter_openat")
2 int valid_example(struct trace_event_raw_sys_enter *ctx)
3 {
4     char comm[16];
5     bpf_get_current_comm(&comm, sizeof(comm));
6     bpf_printk("openat called by %s\n", comm);
7     return 0;
8 }
  
```

### Benefits of the eBPF Verifier

- ✓ Prevents kernel crashes
- ✓ Improves system stability
- ✓ Allows eBPF to run safely in production
- ✓ Enables powerful observability and security tools
- ✓ Protects against malicious or buggy programs
- ✓ Builds trust in eBPF usage



#### Junior Engineer Tip

If your eBPF program fails to load, read the verifier error message carefully. It usually tells you exactly what is wrong, such as an invalid memory access, unbounded loop or unsupported helper call.



#### Remember This

- The verifier is the reason eBPF can run safely inside the kernel.
- Not all valid C code is allowed in eBPF.
- You must follow kernel rules, limits and allowed helper functions.
- Passing the verifier is mandatory before a program can run.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 09 eBPF Maps

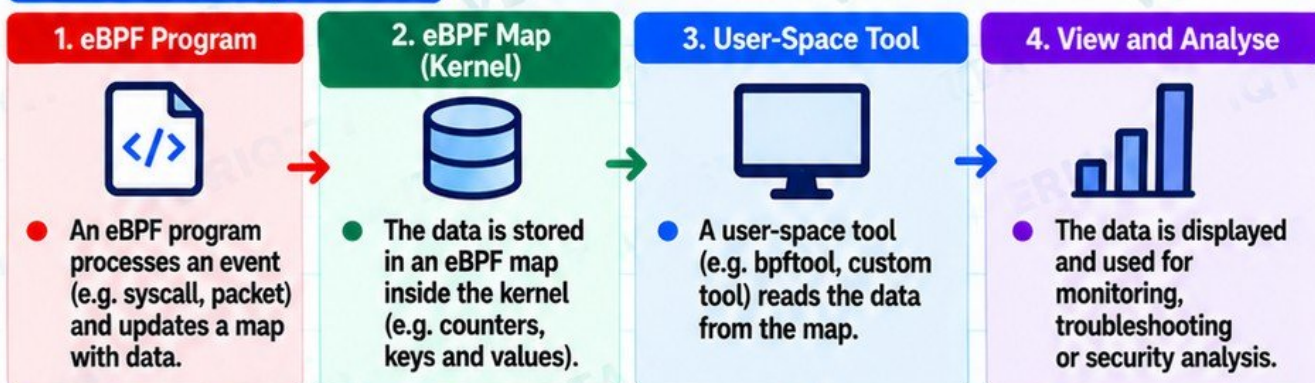
## What Are eBPF Maps?

- eBPF maps are kernel data structures used by eBPF programs to store and share data.
- They allow eBPF programs to keep state, collect statistics and communicate with user space.
- Maps can be read and updated by eBPF programs and by user-space tools.
- They act as a bridge between the kernel and user space.

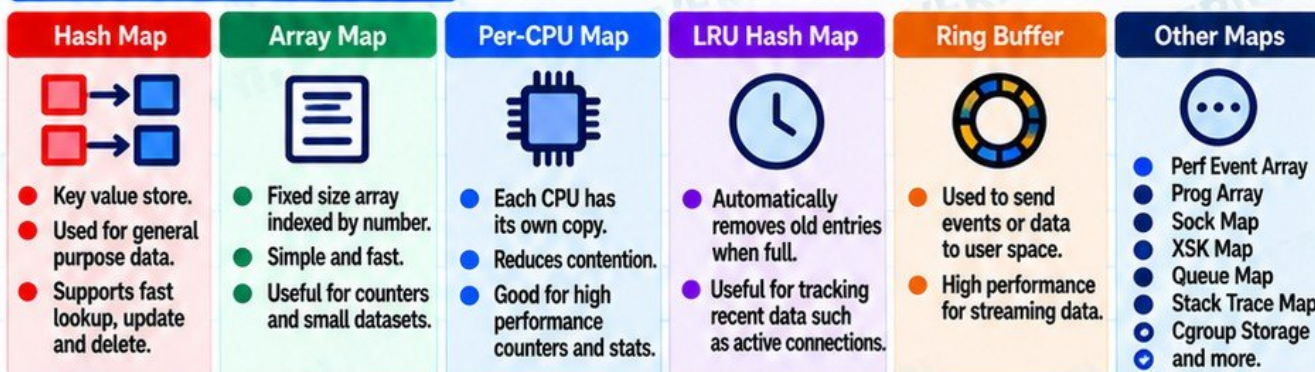
## Why eBPF Maps Are Important?

- They allow eBPF programs to keep state across events.
- They enable collection of metrics, counters and other useful data.
- They allow sharing data between multiple eBPF programs.
- They make it possible for user-space tools to read data from the kernel in real time.
- They are essential for building observability, security and networking tools.

## How eBPF Maps Work



## Common eBPF Map Types



## Example: Counting System Calls with a Map

```

1 struct {
2     __uint(type, BPF_MAP_TYPE_HASH);
3     __type(key, u32);           // syscall ID
4     __type(value, u64);         // counter
5     __uint(max_entries, 1024);
6 } syscall_count SEC(".maps");

7 // In the eBPF program
8 u32 id = bpf_get_current_pid_tgid() >> 32;
9 u64 *count = bpf_map_lookup_elem(&syscall_count, &id);
10 if (count)
11     __sync_fetch_and_add(count, 1);
  
```

## Real World Use Cases

- ✓ Counting system calls
- ✓ Tracking active network connections
- ✓ Collecting per-process statistics
- ✓ Storing security events
- ✓ Sharing data between multiple eBPF programs
- ✓ Sending events to user space tools
- ✓ Building monitoring and observability tools

## Junior Engineer Tip

Choose the right map type based on your use case. For high frequency data, use per-CPU maps or ring buffers for better performance.

## Remember This

- Maps are inside the kernel.
- They have a fixed size.
- You must choose the correct map type for your use case.
- Maps allow eBPF programs and user-space tools to share data safely.
- Using the wrong map type can lead to performance issues or data loss.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 10 From eBPF Program to Insight

Loading, attaching, collecting events and presenting useful information

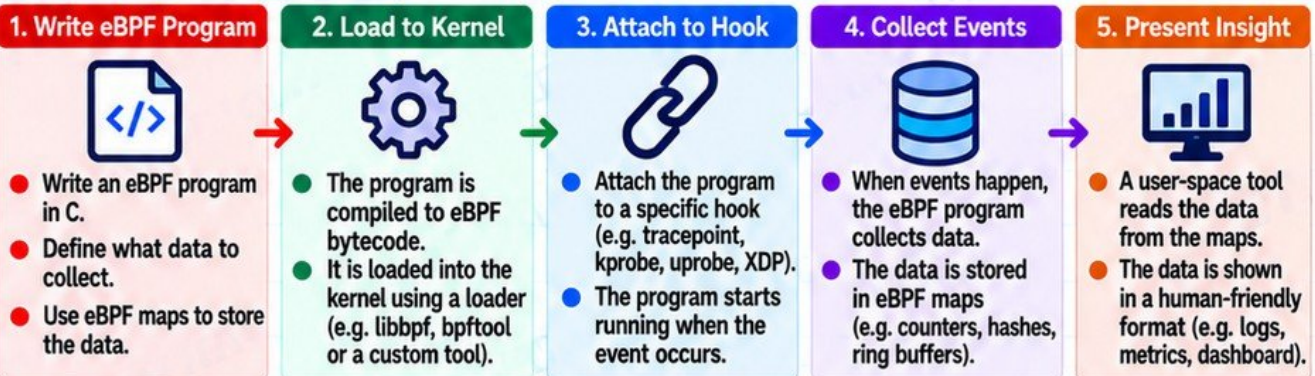
## What Does This Mean?

- An eBPF program is first loaded into the Linux kernel.
- It is then attached to a specific hook (tracepoint, kprobe, uprobe, XDP, etc.).
- When events occur, the program collects relevant data and stores it in eBPF maps.
- A user-space tool reads the data from the maps and presents it in a human-friendly way.
- This workflow turns raw kernel events into useful insights for troubleshooting, monitoring and security.

## Why This Is Important?

- It gives real-time visibility into what is happening inside the kernel.
- It helps you understand the behaviour of applications, network and the system.
- It allows you to detect and troubleshoot problems quickly.
- It can be used for performance monitoring, security analysis and observability.
- It provides deep insights without modifying the kernel or adding heavy overhead.

## The End-to-End Flow



## Example: Monitoring System Calls

### 1. eBPF Program (C)

```

1 #include <linux/bpf.h>
2 #include <bpf/bpf_helpers.h>
3 struct {
4     __uint(type, BPF_MAP_TYPE_HASH);
5     __type(key, u32); // syscall ID
6     __type(value, u64); // counter
7     __uint(max_entries, 1024);
8 } syscall_count SEC(".maps");
9
10 SEC("tracepoint/syscalls/sys_enter_openat")
11 int handle_openat(struct trace_event_raw_sys_enter *ctx)
12 {
13     u32 id = ctx->id;
14     u64 *count;
15     count = bpf_map_lookup_elem(&syscall_count, &id);
16     if (count)
17         __sync_fetch_and_add(count, 1);
18     else {
19         u64 init = 1;
20         bpf_map_update_elem(&syscall_count, &id, &init, BPF_ANY);
21     }
22     return 0;
23 }
  
```

### 2. Load and Attach (using bpftool)

```

# Compile the program
clang -O2 -target bpf -c syscall_monitor.c -o syscall_monitor.o

# Load the program into the kernel
sudo bpftool prog load syscall_monitor.o /sys/fs/bpf/syscall_monitor

# Attach to tracepoint
sudo bpftool prog attach \
    pinned /sys/fs/bpf/syscall_monitor \
    tracepoint syscalls sys_enter_openat
  
```

### 3. Read Data and Show Insight (using bpftool)

```

# Read the map
sudo bpftool map dump pinned /sys/fs/bpf/syscall_count
  
```

```

key: 0x00000101 value: 1234
key: 0x00000103 value: 567
key: 0x00000104 value: 89
key: 0x00000105 value: 4321
  
```

This shows the number of times each syscall was called.

## Junior Engineer Tip

Start with a simple tracepoint program to understand the flow. Use existing tools like bpftool and bcc examples to learn faster.

## Real World Insights

- ✓ Identify performance bottlenecks.
- ✓ Monitor application and system behaviour.
- ✓ Detect unusual or malicious activity.
- ✓ Build custom observability tools.

## Remember This

- You must have the correct permissions (usually root).
- Choose the right hook for your use case.
- Use eBPF maps to store and share data.
- Keep the program safe and efficient.
- Present the data in a clear and useful format.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 11 Your First eBPF Tools

## Introduction to BCC, bpftrace and modern eBPF tooling

### What Are eBPF Tools?

- eBPF tools help you write, load and run eBPF programs without dealing with all the low level details.
- They make it easy to trace the kernel, collect data and get insights in real time.
- Popular tools include BCC, bpftrace and modern eBPF based tools.
- Each tool has a different approach but they all use eBPF under the hood.

### Why Use eBPF Tools?

- They give you deep visibility into the Linux kernel and system behaviour.
- They help you troubleshoot performance, security and networking issues.
- They are safe and efficient because they use eBPF and the kernel verifier.
- They allow you to collect real time data without modifying the kernel or restarting applications.

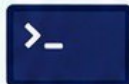
### Popular eBPF Tools and Their Purpose

#### BCC (BPF Compiler Collection)



- A powerful set of tools and libraries for writing eBPF programs.
- Comes with many ready to use tools for tracing, profiling and monitoring.
- Uses a Python or C based approach.
- Good for learning and in depth analysis.

#### bpftrace



##### bpftrace

- A high level and easy to use tracing language for eBPF.
- Simple one line commands for common tracing tasks.
- No need to write C code in most cases.
- Great for quick diagnostics and ad hoc analysis.

#### libbpf and bpftool



##### libbpf / bpftool

- Low level tools and libraries for building production grade eBPF programs.
- bpftool is used to load, inspect and manage eBPF programs, maps and attachments.
- Commonly used in modern eBPF projects and by advanced users.

#### Modern eBPF Tooling



- Includes tools like Cilium, Pixie, Parca, inspektor-gadget and others.
- Built on eBPF to provide advanced observability, security and networking features.
- Often come with a user friendly interface and are designed for real world use.

### Example Commands

#### Using BCC (execsnoop)

```
sudo execsnoop
```

| PID  | PPID | COMM   |
|------|------|--------|
| 1234 | 1    | bash   |
| 2345 | 1234 | python |
| 6789 | 2345 | curl   |
| 7810 | 2345 | ls     |

Shows new processes as they are executed on the system.

#### Using bpftrace (syscalls)

```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat { printf("%d %s\n", pid, comm); }'
```

| PID  | COMMAND    |
|------|------------|
| 1234 | nginx      |
| 2345 | python     |
| 6789 | curl       |
| 7810 | containerd |

Traces openat system calls in real time showing the process ID and command.

#### Using bpftool (list programs)

```
sudo bpftool prog show
```

|      |            |                       |
|------|------------|-----------------------|
| 123: | tracepoint | name sys_enter_openat |
| 124: | kprobe     | name do_sys_open      |
| 125: | xdp        | name xdp_filter       |
| 126: | tracepoint | name sys_exit_openat  |

Lists loaded eBPF programs in the kernel.

### Junior Engineer Tip

Start with bpftrace for simple tracing. Explore BCC tools to see real examples. Use bpftool to understand how eBPF programs are loaded and managed in the kernel.

### Real World Use Cases

- ✓ Troubleshoot application performance
- ✓ Analyse system calls and file access
- ✓ Monitor network activity
- ✓ Detect unusual behaviour
- ✓ Build custom observability tools

### Remember This

- You usually need root privileges.
- Different tools have different strengths.
- Start with simple commands and examples.
- All these tools use eBPF and are verified by the kernel.
- They provide safe and powerful visibility into your system.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 12 Tracing with bpftrace

Reading a simple bpftrace program and understanding probe → action → output

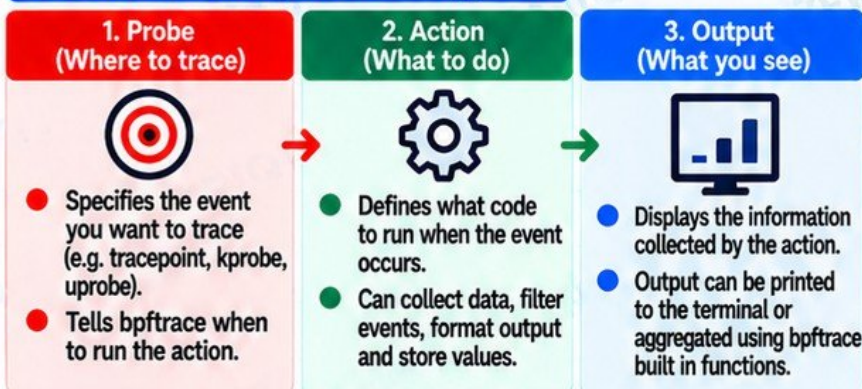
## What Is bpftrace?

- bpftrace is a high level tracing language for eBPF.
- It allows you to trace the Linux kernel and user space with simple one line scripts.
- You can write, run and get useful insights without writing C code.
- It is ideal for learning, troubleshooting and quick analysis.

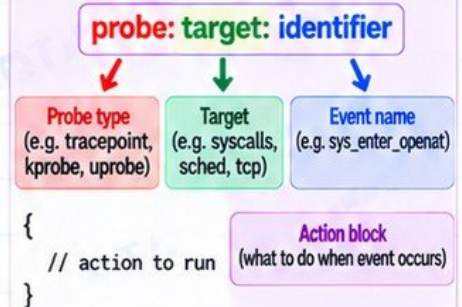
## Why Use bpftrace?

- Simple and easy to learn syntax.
- Allows you to trace real events in the kernel and user space.
- Great for debugging performance, network, I/O, system calls and more.
- Produces useful output in real time.
- Helps you understand how applications interact with the kernel.

## Structure of a bpftrace Program



## Basic bpftrace Syntax



## Example 1: Trace File Open System Calls

### bpftrace Program

```
1 tracepoint:syscalls:sys_enter_openat
2 {
3     printf("%d %s %s\n",
4         pid, comm, str(args->filename));
5 }
```

### Code Explanation

- 1 tracepoint:syscalls:sys\_enter\_openat**  
Traces when a process calls openat.
- 2 printf(...)**  
Prints the process ID, command name and filename.
- 3 pid, comm, str(args->filename)**  
Built in variables that provide information about the event.

### Expected Output (example)

```
1234 nginx /etc/nginx/nginx.conf
2345 python /home/user/app.py
6789 curl /etc/hosts
4321 containerd /var/lib/containerd/config.toml
```

Shows the process ID, command name and the file being opened in real time.

## Example 2: Count System Calls Using a Map

### bpftrace Program

```
1 tracepoint:syscalls:sys_enter_openat
2 {
3     @count[comm] = count();
4 }
5 interval:s:5
6 {
7     print(@count);
8     clear(@count);
9 }
```

### Code Explanation

- 1 @count[comm] = count()**  
Counts the number of openat calls per process name.
- 2 interval:s:5**  
Runs every 5 seconds.
- 3 print(@count)**  
Displays the count for each process.
- 4 clear(@count)**  
Resets the map for the next interval.

### Expected Output (example)

```
Attaching 2 probes...
@[comm] count
nginx 120
python 45
curl 12
containerd 8
(every 5 seconds)
```

Shows the number of openat calls per process every 5 seconds.



## Junior Engineer Tip

- Start with tracepoints, they are stable and easy to use.
- Use built in variables like pid, comm, uid, retval and args.
- Try simple programs first and gradually explore filters, maps and different probe types.



## Real World Use Cases

- ✓ Monitor file, network or process activity.
- ✓ Troubleshoot application behaviour.
- ✓ Identify performance bottlenecks.
- ✓ Understand how applications use system calls.
- ✓ Build real time observability tools.



## Remember This

- Always run bpftrace with root privileges.
- Use tracepoints where possible.
- Be careful with high frequency events.
- Start with simple scripts.
- Use maps to aggregate data for analysis.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 13 Tracing System Calls

Observing what applications ask the Linux kernel to do

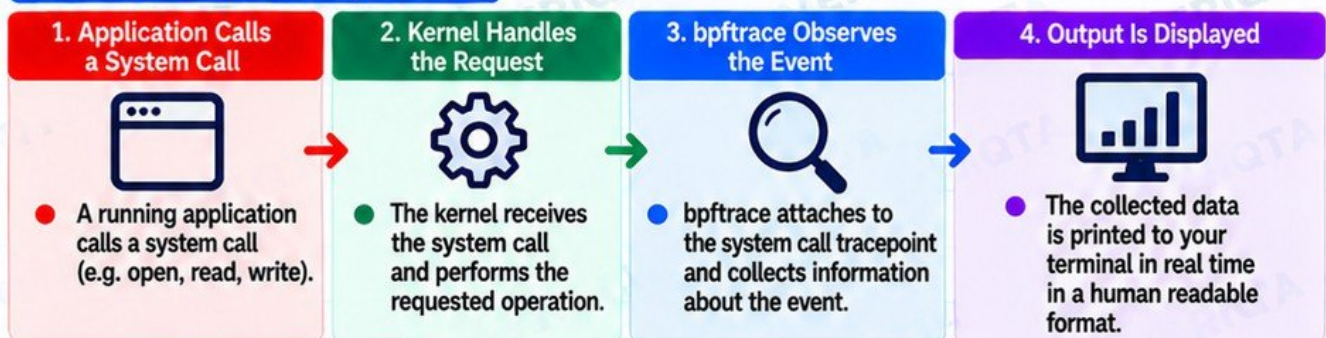
## What Are System Calls?

- System calls are the interface between user space applications and the Linux kernel.
- Applications use system calls to perform tasks such as reading files, writing files, creating processes and accessing the network.
- Every important action an application takes usually goes through a system call.
- Tracing system calls helps you understand what an application is doing in real time.

## Why Trace System Calls?

- Helps you see the real behaviour of applications.
- Useful for troubleshooting performance, errors and unexpected behaviour.
- Helps you understand file access, network activity, process creation and more.
- Useful for security analysis and incident investigation.
- Shows you what is actually happening instead of guessing.

## How It Works with bpftrace



### Example 1: Trace File Open Calls

#### bpftrace Program

```

1 tracepoint:syscalls:sys_enter_openat
2 {
3     printf("%s PID=%d openat %s\n",
4           comm, pid, str(args->filename));
5 }
  
```

#### What It Shows

- Monitors the openat system call.
- Shows the process name (comm).
- Shows the process ID (pid).
- Shows the file path being opened (args->filename).

#### Example Output

```

nginx PID=1234 openat /etc/nginx/nginx.conf
python PID=2345 openat /home/user/app.py
curl PID=6789 openat /etc/hosts
bash PID=7810 openat /usr/share/zoneinfo/UTC
  
```

### Example 2: Trace Read and Write Calls

#### bpftrace Program

```

1 tracepoint:syscalls:sys_enter_read,
2 tracepoint:syscalls:sys_enter_write
3 {
4     printf("%s PID=%d %s fd=%d count=%d\n",
5           comm, pid, probe, args->fd, args->count);
6 }
7
  
```

#### What It Shows

- Monitors read and write system calls.
- Shows which process is reading or writing.
- Displays the file descriptor (fd).
- Shows the number of bytes requested (count).

#### Example Output

```

python PID=2345 sys_enter_read fd=3 count=4096
python PID=2345 sys_enter_write fd=1 count=23
nginx PID=1234 sys_enter_read fd=5 count=8192
curl PID=6789 sys_enter_write fd=1 count=512
  
```

### Example 3: Trace Execve (Process Execution)

#### bpftrace Program

```

1 tracepoint:syscalls:sys_enter_execve
2 {
3     printf("%s PID=%d execve %s\n",
4           comm, pid, str(args->filename));
5 }
  
```

#### What It Shows

- Monitors the execve system call.
- Shows when a new process is executed.
- Displays the process name, PID and the executable path.

#### Example Output

```

bash PID=7810 execve /usr/bin/ls
python PID=2345 execve /usr/bin/python3
curl PID=6789 execve /usr/bin/curl
systemd PID=1 execve /usr/lib/systemd/systemd
  
```

## Junior Engineer Tip

- Start by tracing one system call at a time.
- Use common calls like openat, read, write and execve.
- Look at the output and relate it to what the application is doing.
- This is a powerful way to understand application behaviour.

## Real World Use Cases

- ✓ Troubleshoot application issues.
- ✓ Identify unexpected file or network access.
- ✓ Analyse application performance.
- ✓ Investigate suspicious behaviour.
- ✓ Understand how applications use the kernel.

## Remember This

- You usually need root privileges.
- System call tracing can generate a lot of data.
- Use filters to focus on specific processes.
- Be careful when tracing in production.
- Use the information responsibly.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 14 Tracing Processes

## Process creation, execution and termination

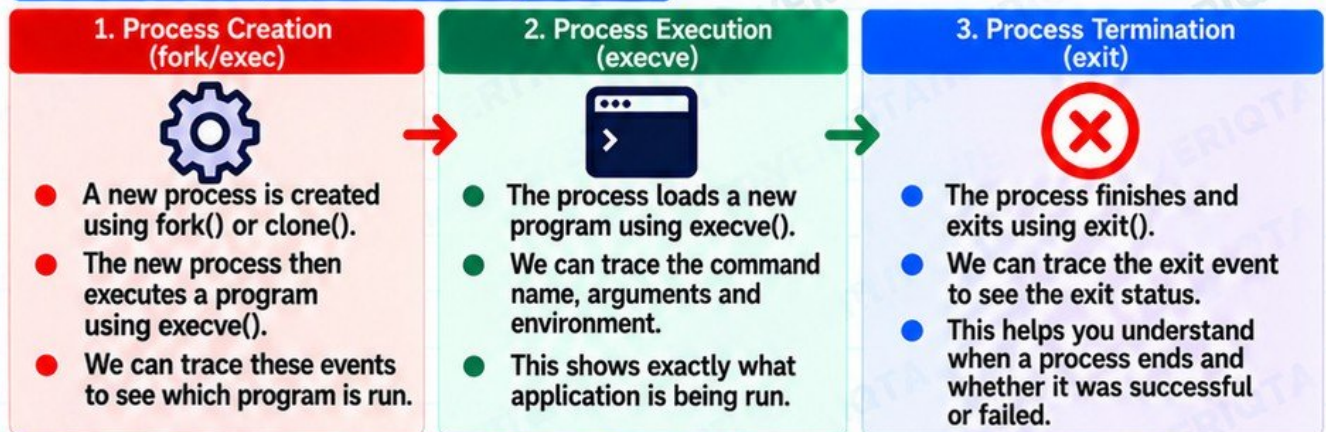
### What Does This Mean?

- Processes are created, execute tasks and eventually terminate.
- Each of these actions involves system calls and kernel events.
- Tracing processes helps you see what applications are running and what they do in real time.
- It is useful for troubleshooting, security analysis and understanding system behaviour.

### Why Trace Processes?

- Helps you see when new processes are created and by which user.
- Shows the commands that applications execute.
- Helps you understand process relationships (parent and child).
- Useful for detecting unexpected or suspicious processes.
- Provides visibility into when processes exit and their exit status.

### Process Lifecycle and Tracing Points



### Example 1: Trace Process Execution (execve)

#### bpftrace Program

```
1 tracepoint:syscalls:sys_enter_execve
2 {
3     printf("%s PID=%d %s\n",
4           comm, pid, str(args->filename));
5 }
```

#### What It Shows

- Traces when a process executes a new program.
- Displays the process name, PID and the file being executed.

#### Example Output

```
bash PID=1234 /usr/bin/python3
nginx PID=2345 /usr/sbin/nginx
curl PID=6789 /usr/bin/curl
systemd PID=1 /usr/lib/systemd/systemd
```

### Example 2: Trace Process Creation (fork/clone)

#### bpftrace Program

```
1 tracepoint:syscalls:sys_enter_clone
2 {
3     printf("%s PID=%d created a new process\n",
4           comm, pid);
5 }
```

#### What It Shows

- Traces when a process creates a new process.
- Shows the parent process name and PID.

#### Example Output

```
bash PID=1234 created a new process
nginx PID=2345 created a new process
python PID=6789 created a new process
sshd PID=1001 created a new process
```

### Example 3: Trace Process Termination (exit)

#### bpftrace Program

```
1 tracepoint:syscalls:sys_exit_exit
2 {
3     printf("%s PID=%d exited with code %d\n",
4           comm, pid, args->ret);
5 }
```

#### What It Shows

- Traces when a process exits.
- Displays the process name, PID and exit code.

#### Example Output

```
python PID=6789 exited with code 0
curl PID=4321 exited with code 1
nginx PID=2345 exited with code 0
bash PID=1234 exited with code 0
```

### Junior Engineer Tip

- Start with these simple traces to understand process behaviour.
- Look at process names, arguments and exit codes.
- Try running common commands and see the output.

### Real World Use Cases

- Investigate unexpected processes.
- Monitor what users are executing.
- Detect suspicious activity.
- Understand application start up behaviour.
- Troubleshoot failed executions.

### Remember This

- You usually need root privileges.
- Process tracing can generate a lot of data.
- Be careful when tracing in production.
- Use filters to focus on specific processes.
- Always analyse the information responsibly.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 15 Tracing File Activity

Discovering which processes access files and where filesystem activity originates

## What Is File Activity Tracing?

- File activity tracing shows which processes access files on your system.
- It helps you see when files are opened, read, written, created, deleted or modified.
- It combines process information with filesystem events so you can understand the source of the activity.
- It is useful for troubleshooting, security analysis and understanding application behaviour.

## Why Trace File Activity?

- Helps you identify which process is reading or writing a specific file.
- Useful for investigating unexpected changes, suspicious activity or performance issues.
- Helps you understand how applications use the filesystem.
- Shows the full path of files being accessed.
- Useful for monitoring configuration files, log files and important system files.

## Common File Operations to Trace

### 1. Open (openat)



- Occurs when a file is opened.
- Shows which process opens a file.
- Provides the file path and open flags.

### 2. Read (read)



- Occurs when data is read from a file.
- Helps you see which process reads files.
- Useful for tracking access to sensitive files.

### 3. Write (write)



- Occurs when data is written to a file.
- Shows which process modifies a file.
- Helps you detect unexpected or suspicious changes.

### 4. Close (close)



- Occurs when a file is closed.
- Helps you understand when a process finishes using a file.

### 5. Other Events



- Create (creat)
- Delete (unlink)
- Rename (rename)
- Metadata changes (chmod, chown)
- Directory operations (mkdir, rmdir)

## Example 1: Trace File Opens (openat)

### bpftrace Program

```
1 tracepoint:syscalls:sys_enter_openat
2 {
3     printf("%s PID=%d open %s\n",
4           comm, pid, str(args->filename));
5 }
```

### What It Shows

- Shows when a process opens a file.
- Displays the process name (comm).
- Shows the process ID (PID).
- Displays the full file path.

### Example Output

```
nginx 1234 open /etc/nginx/nginx.conf
python 2345 open /home/user/app.py
bash 6789 open /var/log/syslog
systemd 1 open /etc/passwd
```

## Example 2: Trace File Writes

### bpftrace Program

```
1 tracepoint:syscalls:sys_enter_write
2 {
3     printf("%s PID=%d write fd=%d\n",
4           comm, pid, args->fd);
5 }
```

### What It Shows

- Shows when a process writes to a file.
- Displays the process name and PID.
- Shows the file descriptor being written.
- Helps you identify which process is modifying files.

### Example Output

```
python 2345 write fd=1
nginx 1234 write fd=3
bash 6789 write fd=2
docker 4321 write fd=5
```

## Example 3: Trace File Deletions (unlink)

### bpftrace Program

```
1 tracepoint:syscalls:sys_enter_unlinkat
2 {
3     printf("%s PID=%d delete %s\n",
4           comm, pid, str(args->pathname));
5 }
```

### What It Shows

- Shows when a process deletes a file.
- Displays the process name and PID.
- Shows the full path of the file being deleted.
- Useful for monitoring unexpected deletions.

### Example Output

```
rm 5678 delete /tmp/test.log
bash 6789 delete /home/user/file.txt
python 2345 delete /var/cache/data.tmp
nginx 1234 delete /var/www/index.html
```

## Junior Engineer Tip

- Start by tracing file opens to understand which processes access important files.
- Look at the process name, PID and full file path.
- Combine different file events to get a complete picture.

## Real World Use Cases

- ✓ Identify which process is modifying configuration files.
- ✓ Investigate unexpected file changes.
- ✓ Monitor access to sensitive files.
- ✓ Troubleshoot application file errors.
- ✓ Detect suspicious or malicious activity.

## Remember This

- You usually need root privileges.
- File activity can generate a lot of data.
- Use filters to focus on specific files or processes.
- Be careful when tracing in production.
- Always analyse the information responsibly.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 16 Troubleshooting CPU with eBPF

Finding CPU-heavy execution paths rather than stopping at "CPU is high"

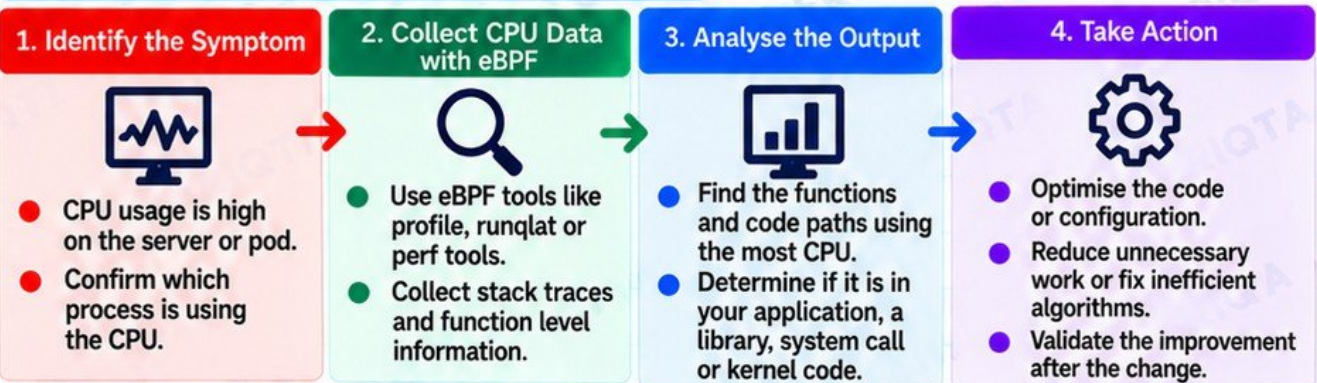
## What Does This Mean?

- eBPF allows you to see exactly where CPU time is being spent inside the kernel and user applications.
- You can identify the functions, system calls and code paths that are consuming CPU.
- Instead of just knowing that CPU is high, you get detailed visibility into why it is high.
- This helps you focus on the real cause and take the right action.

## Why Use eBPF for CPU Troubleshooting?

- Shows real time CPU hotspots at the function and stack level.
- Helps you find which process and thread is using the CPU.
- Identifies whether the CPU time is in kernel code, user code or system calls.
- Useful for performance tuning, troubleshooting and capacity planning.
- Helps you understand application behaviour in production.

## How to Find CPU Heavy Execution Paths



### Example 1: Find CPU Hotspots with bpftrace

| bpftrace Program                                                           | What It Shows                                                                                                                                                                                                       | Example Output (top entries)                                                                                                                                        |
|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 1 profile:cpu:99 2 { 3     @[ustack] = count(); 4 }           </pre> | <ul style="list-style-type: none"> <li>● Samples the CPU stack at 99Hz.</li> <li>● Shows which user space functions are using the most CPU.</li> <li>● Helps identify hot functions and execution paths.</li> </ul> | <pre> @[ustack] 10234 myapp::process_data 5234  myapp::calculate 4321  libc::malloc 3210  libstdc++::vector::push_back 2100  myapp::handle_request           </pre> |

### Example 2: Compare On CPU vs Off CPU Time

| bpftrace Program                                                                                                                                            | What It Shows                                                                                                                                                                                                                                                              | Example Output (on CPU)                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 1 profile:hz:99 { @[ustack] = count(); } 2 3 tracepoint:sched:sched_switch 4 /args-&gt;prev_state != 0/ 5 { @[ustack] = count(); } 6           </pre> | <ul style="list-style-type: none"> <li>● Shows where CPU time is being spent (on CPU).</li> <li>● Also shows where the application is waiting (off CPU).</li> <li>● Helps you understand if the issue is CPU bound or waiting on I/O, locks or other resources.</li> </ul> | <pre> @[ustack] count myapp::process_data 10234 myapp::calculate    5234 libc::malloc        4321 kernel::tcp_recvmg  2100           </pre> |

### Example 3: Trace Specific Process CPU Usage

| bpftrace Program                                                                | What It Shows                                                                                                                                                                                                                               | Example Output (top entries)                                                                                                                       |
|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 1 profile:pid:1234:99 2 { 3     @[ustack] = count(); 4 }           </pre> | <ul style="list-style-type: none"> <li>● Profiles CPU usage for a specific process using its PID.</li> <li>● Useful when you want to focus on one application.</li> <li>● Shows the functions consuming CPU inside that process.</li> </ul> | <pre> @[ustack] count myapp::handle_request 6321 myapp::process_data   4210 libc::poll            3120 libc::read            2100           </pre> |

### Junior Engineer Tip

- Start with a broad view, then narrow down.
- Use on CPU and off CPU analysis.
- Always look at stack traces to find the real cause.
- Validate the fix with the same eBPF tool.

### Real World Use Cases

- ✓ Investigate high CPU usage in production.
- ✓ Find inefficient code or busy loops.
- ✓ Understand performance bottlenecks.
- ✓ Troubleshoot slow applications.
- ✓ Optimise application and system performance.

### Remember This

- High CPU does not always mean a bug in your code.
- The CPU time can be in the kernel, libraries or system calls.
- Use eBPF to get evidence, not guess.
- Always analyse the full context before taking action.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 17 Troubleshooting Memory

Observing allocation, page faults and memory-related behavior without oversimplifying memory diagnosis

## What Does This Mean?

- eBPF allows you to observe memory behaviour inside the kernel and user applications in real time.
- You can see memory allocations, page faults, memory mappings and release of memory.
- This helps you understand why memory usage is high instead of just looking at total memory usage.
- It gives you visibility into which processes, functions or code paths are responsible.

## Why Use eBPF for Memory Troubleshooting?

- Helps you identify which process or code path is allocating memory.
- Shows when and why page faults are happening.
- Useful for detecting memory leaks or unexpected growth.
- Helps you understand memory behaviour at the system call and kernel level.
- Gives real time insights without restarting applications.

## Key Memory Events You Can Observe

### 1. Memory Allocation (malloc, mmap)



- Tracks when memory is allocated.
- Shows the size of the allocation.
- Helps you find which process or library is allocating memory.

### 2. Page Faults



- Occurs when a page is not in memory.
- Shows whether it is a major or minor fault.
- Helps you understand memory pressure and I/O activity.

### 3. Memory Mapping (mmap)



- Shows when files or anonymous memory are mapped.
- Displays the memory protection and size.
- Helps you understand how applications use virtual memory.

### 4. Memory Release (free, munmap)



- Tracks when memory is released.
- Helps you detect if memory is not being freed properly.
- Useful for identifying memory leaks.

## Example 1: Trace Memory Allocations (malloc)

### bpfftrace Program

```
1 uprobe:/lib/x86_64-linux-gnu/libc.so.6:malloc
2 {
3     printf("%s PID=%d allocated %d bytes\n",
4           comm, pid, arg0);
5 }
```

### What It Shows

- Shows when a process allocates memory using malloc.
- Displays the process name, PID and size of the allocation.
- Helps you identify which application is allocating memory.

### Example Output

```
nginx PID=1234 allocated 4096 bytes
python PID=2345 allocated 8192 bytes
java PID=6789 allocated 16384 bytes
myapp PID=4321 allocated 1024 bytes
```

## Example 2: Trace Page Faults

### bpfftrace Program

```
1 tracepoint:exceptions:page_fault_user
2 {
3     printf("%s PID=%d page fault=%d address=%p\n",
4           comm, pid, args->address);
5 }
```

### What It Shows

- Shows when a user space page fault occurs.
- Displays the process name, PID and the memory address.
- Helps you identify applications that are causing page faults.

### Example Output

```
nginx PID=1234 page fault address=0x7f...
python PID=2345 page fault address=0x7a...
java PID=6789 page fault address=0x7b...
myapp PID=4321 page fault address=0x7c...
```

## Example 3: Trace Memory Mapping (mmap)

### bpfftrace Program

```
1 tracepoint:syscalls:sys_enter_mmap
2 {
3     printf("%s PID=%d mmap size=%d prot=%d\n",
4           comm, pid, args->len, args->prot);
5 }
```

### What It Shows

- Shows when a process maps memory using mmap.
- Displays the process name, PID, size and protection flags.
- Helps you understand how applications use virtual memory.

### Example Output

```
nginx PID=1234 mmap size=4096 prot=3
python PID=2345 mmap size=8192 prot=1
java PID=6789 mmap size=65536 prot=3
myapp PID=4321 mmap size=12288 prot=3
```

## Junior Engineer Tip

- Look at allocations, page faults and mappings together.
- Focus on the process name, PID and size.
- Compare the data over time to find unusual patterns.
- Use multiple tools to get a complete picture.

## Real World Use Cases

- ✓ Investigate high memory usage in production.
- ✓ Find memory leaks in applications.
- ✓ Identify processes causing frequent page faults.
- ✓ Understand memory mapping behaviour.
- ✓ Troubleshoot out of memory issues.

## Remember This

- High memory usage does not always mean a leak.
- Page faults can be normal or a sign of memory pressure.
- Consider the full context before taking action.
- Use eBPF data together with other monitoring tools.
- Always verify the findings in a safe environment.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 18 Troubleshooting Disk I/O

Finding slow I/O, latency and processes responsible for storage activity

## What Does This Mean?

- Disk I/O tracing shows which processes are reading from or writing to storage and how long the I/O takes.
- It helps you identify slow I/O operations, high latency and processes responsible for storage activity.
- You can see the file paths, block devices and I/O patterns causing the issue.
- This gives you real evidence to troubleshoot performance problems instead of guessing.

## Why Use eBPF for Disk I/O Troubleshooting?

- Helps you find which process is causing high I/O or slow disk access.
- Shows read and write latency in real time.
- Helps you understand whether the issue is the application, filesystem or storage device.
- Useful for detecting unexpected or excessive I/O activity.
- Provides visibility into I/O patterns without restarting applications.

## Common Disk I/O Issues You Can Investigate

### 1. High I/O Activity



- Identify which processes are performing disk I/O.
- See read and write rates.
- Useful when the disk is busy and applications are slow.

### 2. High I/O Latency



- Find I/O operations that take a long time to complete.
- Identify whether reads or writes are slow.
- Helps you understand where the delay is coming from.

### 3. Identify Access Patterns



- See which files are being read or written.
- Check the size and frequency of I/O.
- Helps you detect unusual or unexpected file access patterns.

### 4. Find the Responsible Process



- Identify the process name, PID and command.
- Useful for stopping unnecessary or harmful I/O activity.
- Helps you take the right action based on real data.

## Example 1: Trace Disk Read and Write Activity

### bpfftrace Program

```
1 tracepoint:block:block_rq_issue
2 {
3     printf("%s PID=%d %s %d bytes\n",
4           comm, pid, args->rwbs, args->bytes);
5 }
```

### What It Shows

- Shows when a process issues a disk read or write request.
- Displays the process name, PID, operation type and size.
- Helps you see which processes are performing I/O and how much.

### Example Output

```
nginx PID=1234 W 4096 bytes
mysql PID=2345 R 8192 bytes
java PID=6789 W 32768 bytes
python PID=4321 R 4096 bytes
```

## Example 2: Trace Disk I/O Latency

### bpfftrace Program

```
1 tracepoint:block:block_rq_complete
2 {
3     $delta = nsecs - args->start_time_ns;
4     printf("%s PID=%d %s latency=%d us\n",
5           comm, pid, args->rwbs, $delta / 1000);
6 }
```

### What It Shows

- Shows how long each I/O request takes to complete.
- Displays the process name, PID, operation type and latency.
- Helps you identify slow I/O operations and high latency.

### Example Output

```
nginx PID=1234 R latency=230 us
mysql PID=2345 W latency=1520 us
java PID=6789 R latency=980 us
python PID=4321 W latency=310 us
```

## Example 3: Trace File I/O with Process and Path

### bpfftrace Program

```
1 tracepoint:syscalls:sys_enter_openat
2 {
3     printf("%s PID=%d open %s\n",
4           comm, pid, atr(args->filename));
5 }
```

### What It Shows

- Shows when a process opens a file on the filesystem.
- Displays the process name, PID and full file path.
- Helps you understand which files are being accessed and by which process.

### Example Output

```
nginx PID=1234 open /var/log/nginx/access.log
mysql PID=2345 open /var/lib/mysql/ibdata1
java PID=6789 open /opt/app/data.db
python PID=4321 open /tmp/cache/file.tmp
```

## Junior Engineer Tip

- Start with general disk I/O tracing, then narrow down.
- Look at the process name, PID, file path and latency.
- Compare read and write patterns.
- Use multiple traces to understand the full picture.

## Real World Use Cases

- ✓ Investigate slow application performance.
- ✓ Identify which process is causing high disk I/O.
- ✓ Detect unusual or unexpected file access.
- ✓ Troubleshoot database or log write latency.
- ✓ Validate storage performance and behaviour.

## Remember This

- High disk I/O does not always mean a faulty disk.
- Slow I/O can be caused by the application, filesystem or storage device.
- Always look at the full context before taking action.
- Use eBPF data to confirm the real cause.
- Monitor both reads and writes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 19 Troubleshooting Network Connections

Seeing TCP connections, connection attempts, latency and failures

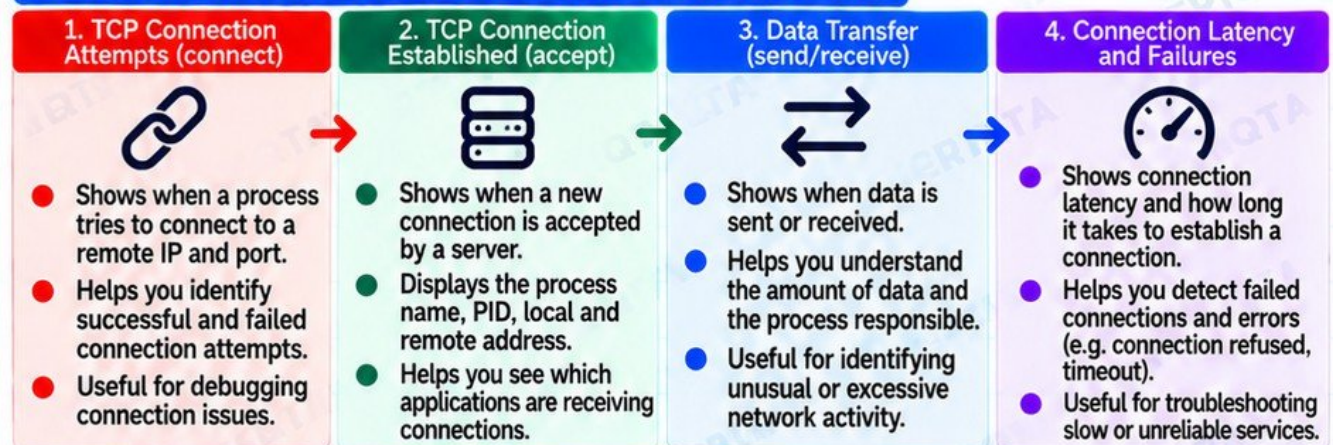
## What Does This Mean?

- Network connection tracing shows when applications create, connect, send, receive and close TCP connections.
- It helps you see connection attempts, latency and failures in real time.
- You can identify which processes are connecting to which IP addresses and ports.
- This gives you real visibility into network behaviour instead of just guessing.

## Why Use eBPF for Network Troubleshooting?

- Shows real time connection attempts, errors and latency.
- Helps you identify which process is making a connection.
- Useful for debugging slow applications and failed connections.
- Helps you understand network behaviour inside the kernel.
- Works without modifying the application or capturing large packet data.

## Common Network Connection Events You Can Trace



### Example 1: Trace TCP Connection Attempts (connect)

#### bpftrace Program

```
1 tracepoint:syscalls:sys_enter_connect
2 {
3     printf("%s PID=%d %s -> %s:%d\n",
4           comm, pid, str(args->uaddr),
5           ntohs(args->addr), ntohs(args->port));
6 }
```

#### What It Shows

- Shows when a process tries to connect to a remote server.
- Displays the process name, PID, remote IP and port.
- Helps you identify successful and failed connection attempts.

#### Example Output

|        |          |                 |
|--------|----------|-----------------|
| nginx  | PID=1234 | 10.0.0.5:443    |
| python | PID=2345 | 192.168.1.20:80 |
| curl   | PID=6789 | 52.84.12.3:443  |
| myapp  | PID=4321 | 10.1.1.10:3306  |

### Example 2: Trace TCP Connection Latency

#### bpftrace Program

```
1 tracepoint:tcp:tcp_connect
2 {
3     printf("%s PID=%d %s:%d latency=%d us\n",
4           comm, pid, ntohs(args->daddr),
5           ntohs(args->dport), args->__ret);
6 }
```

#### What It Shows

- Shows how long a connection takes to establish.
- Displays the process name, PID, remote IP, port and latency.
- Helps you identify slow connections.

#### Example Output

|        |          |                 |         |
|--------|----------|-----------------|---------|
| nginx  | PID=1234 | 10.0.0.5:443    | 320 us  |
| python | PID=2345 | 192.168.1.20:80 | 1450 us |
| curl   | PID=6789 | 52.84.12.3:443  | 980 us  |
| myapp  | PID=4321 | 10.1.1.10:3306  | 4120 us |

### Example 3: Trace Failed TCP Connections

#### bpftrace Program

```
1 tracepoint:tcp:tcp_connect
2 /args->__ret != 0/
3 {
4     printf("%s PID=%d failed to connect %s:%d error=%d\n",
5           comm, pid, ntohs(args->daddr),
6           ntohs(args->dport), args->__ret);
7 }
```

#### What It Shows

- Shows failed connection attempts.
- Displays the process name, PID, remote IP, port and error code.
- Helps you identify connection refused, timeout and other network errors.

#### Example Output

|        |          |                 |            |
|--------|----------|-----------------|------------|
| python | PID=2345 | 10.0.0.5:443    | error=-111 |
| curl   | PID=6789 | 192.168.1.10:80 | error=-110 |
| myapp  | PID=4321 | 10.1.1.20:3306  | error=-111 |
| nginx  | PID=1234 | 52.84.12.3:443  | error=-110 |

## Junior Engineer Tip

- Start by tracing connection attempts and failures.
- Look at the process name, PID and destination IP.
- Compare success and failure patterns.
- Use multiple traces to get the full picture.

## Real World Use Cases

- ✓ Debug slow application connections.
- ✓ Identify unexpected outbound connections.
- ✓ Find processes causing connection failures.
- ✓ Investigate high latency to external services.
- ✓ Monitor service to service communication.

## Remember This

- A failed connection does not always mean the service is down.
- High latency can be caused by network, firewall or the remote server.
- Consider the full context before taking action.
- Always analyse patterns over time.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 20 Finding Hidden Latency

Following evidence across application, kernel, network and storage boundaries

## What Does This Mean?

- Hidden latency is time spent waiting inside different layers such as the application, kernel, network or storage.
- eBPF helps you follow the full request path to see where the delay is actually happening.
- You can see which functions, system calls, network operations or disk operations are taking time.
- This helps you find the real cause instead of guessing or only looking at the application.

## Why Use eBPF for Finding Hidden Latency?

- Gives end to end visibility across application, kernel, network and storage.
- Shows the exact operation that is taking time.
- Helps you identify unexpected delays and performance bottlenecks.
- Useful for troubleshooting slow requests, timeouts and user facing latency issues.
- Helps you understand dependencies and how different components affect each other.

## Typical Sources of Hidden Latency

### Application Layer



- Slow code execution
- Waiting on a database
- External API calls
- Inefficient logic or locks

### Kernel Layer



- System call delays
- Scheduling delays
- Lock contention
- Page faults
- Context switching

### Network Layer



- DNS resolution time
- TCP connection latency
- Packet retransmissions
- Network congestion
- Slow remote service

### Storage Layer



- Slow disk reads or writes
- High I/O latency
- Filesystem metadata operations
- Storage device congestion
- Network storage latency (NFS, EBS, etc)

## Common eBPF Tracing Areas for Latency

### 1. Function Latency



- Measure how long specific functions take.
- Identify slow code paths.

### 2. System Call Latency



- Measure time spent in system calls.
- Find slow I/O or network operations.

### 3. Network Latency



- Trace connection time, request and response time.
- Identify retransmissions and delays.

### 4. Storage Latency



- Measure disk read and write latency.
- Identify slow devices or files.

## Example 1: Trace Function Latency with bpftrace

### bpftrace Program

```
1 uprobe:/usr/bin/myapp:process_request
2 {
3     $start = nsecs;
4 }
5 uretprobe:/usr/bin/myapp:process_request
6 {
7     $lat = (nsecs - $start) / 1000000;
8     printf("latency=%d ms\n", $lat);
9 }
```

### What It Shows

- Measures how long the process\_request function takes.
- Displays the latency in milliseconds.
- Helps you identify slow application code paths.

### Example Output

```
latency=12 ms
latency=8 ms
latency=145 ms
latency=20 ms
latency=310 ms
latency=18 ms
```

## Example 2: Trace End to End Request Latency (Application → Kernel → Network → Storage)

### bpftrace Program

```
1 tracepoint:syscalls:sys_enter_openat
2 {
3     @start[tid] = nsecs;
4 }
5 tracepoint:syscalls:sys_exit_openat
6 /@start[tid]/
7 {
8     $lat = (nsecs - @start[tid]) / 1000000;
9     printf("openat latency=%d ms\n", $lat);
10    delete(@start[tid]);
11 }
```

### What It Shows

- Measures the time taken for file open operations.
- Shows latency for each openat system call.
- Helps you find slow filesystem or storage related delays.

### Example Output

```
openat latency=2 ms
openat latency=18 ms
openat latency=250 ms
openat latency=6 ms
openat latency=420 ms
openat latency=3 ms
```

## Junior Engineer Tip

- Follow the request from application to kernel, then to network and storage.
- Look at multiple layers before making conclusions.
- Use eBPF to get real evidence, not assumptions.

## Real World Use Cases

- ✓ Investigate slow API or web requests.
- ✓ Find latency caused by database, network or storage.
- ✓ Troubleshoot timeouts and failed requests.
- ✓ Understand performance bottlenecks.
- ✓ Optimise application and system performance.

## Remember This

- Latency can come from different layers.
- The slowest part is not always the application code.
- Always verify the evidence across all layers.
- Use eBPF to measure, not guess.
- Consider the full request path before taking action.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 21 eBPF and Containers

## How host-kernel visibility relates to containerized workloads

### What Does This Mean?

- eBPF runs in the Linux kernel and can see events from both host and containerized workloads.
- Containers share the host kernel, so eBPF can observe container processes, network activity, file access and more.
- You can map kernel events to container names, IDs, namespaces and cgroups.
- This gives you real visibility into what containers are doing without changing the application or container image.

### Why Use eBPF for Containers?

- Shows real time visibility into container behaviour and performance.
- Helps you troubleshoot issues inside containers using host level data.
- Lets you see which container is generating CPU, memory, disk or network activity.
- Useful for security, monitoring and incident investigation.
- Works without installing agents inside the container.

### How eBPF Sees Containers

#### 1. Container Runs a Process



- A process runs inside a container.
- The process uses the host kernel.
- All system calls and kernel events are visible to eBPF.

#### 2. Kernel Events Are Traced



- eBPF traces system calls, network events, file access and more.
- Events include process ID, cgroup, namespace and container ID.

#### 3. Events Are Mapped to Containers



- eBPF maps kernel events to container metadata using cgroups and namespaces.
- You can identify the container name, ID, image and pod (in Kubernetes).

#### 4. You Get Container Visibility



- See which container is using CPU, memory, disk or network.
- Identify abnormal behaviour.
- Troubleshoot issues across containers and the host.

### Example 1: See Running Containers and Their Processes

#### bpfftrace Program

```
1 tracepoint:sched:sched_process_exec
2 {
3     printf("%s pid=%d container=%s\n",
4           comm, pid, cgroup);
5 }
```

#### What It Shows

- Shows when a new process starts inside a container.
- Displays the process name, PID and container information.
- Helps you see which container is running which process.

#### Example Output

```
nginx pid=1234 container=nginx-abc123
python pid=2345 container=api-def456
java pid=6789 container=worker-ghi789
redis pid=4321 container=cache-jkl012
```

### Example 2: Trace Container Network Activity

#### bpfftrace Program

```
1 tracepoint:tcp:tcp_connect
2 {
3     printf("%s pid=%d container=%s %s\n",
4           comm, pid, cgroup,
5           ntop(args->daddr), args->dport);
6 }
```

#### What It Shows

- Shows outgoing TCP connections from containers.
- Displays the process name, PID, container and destination IP and port.
- Helps you identify which container is making network connections.

#### Example Output

```
curl pid=1111 container=web-abc123 10.0.0.5:443
nginx pid=2222 container=nginx-abc123 192.168.1.20:80
python pid=3333 container=api-def456 52.84.12.3:443
```

### Example 3: Trace Container File Access

#### bpfftrace Program

```
1 tracepoint:syscalls:sys_enter_openat
2 {
3     printf("%s pid=%d container=%s %s\n",
4           comm, pid, cgroup, str(args->filename));
5 }
```

#### What It Shows

- Shows which files containers are opening or reading.
- Displays the process name, PID, container and file path.
- Helps you identify unexpected file access or configuration reads.

#### Example Output

```
nginx pid=1234 container=nginx-abc123 /etc/nginx/nginx.conf
python pid=2345 container=api-def456 /app/config.yaml
java pid=6789 container=worker-ghi789 /var/log/app.log
```



#### Junior Engineer Tip

- Use container name or ID to filter your traces.
- Combine process, network, file and resource traces for a complete picture.
- Look at both the application and the host kernel events.



#### Real World Use Cases

- ✓ Troubleshoot performance issues in containers.
- ✓ Investigate unexpected network connections.
- ✓ Identify which container is causing high CPU or I/O.
- ✓ Detect suspicious file access or behaviour.
- ✓ Improve monitoring and security visibility.



#### Remember This

- Containers use the host kernel, so eBPF can see their activity.
- Container IDs and names are mapped using cgroups and namespaces.
- Always consider the full context across application, container and host.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 22 eBPF and Kubernetes

Why eBPF has become important for Kubernetes networking, observability and security

## What Does This Mean?

- eBPF runs in the Linux kernel and provides deep visibility into Kubernetes networking, observability and security.
- It allows you to see what is happening inside the cluster without modifying applications or Kubernetes components.
- eBPF can observe pod to pod traffic, service connections, DNS queries, system calls and more.
- This helps you understand, troubleshoot and secure Kubernetes workloads in real time.

## Why Use eBPF for Kubernetes?

- Gives real time visibility into cluster networking and application behaviour.
- Helps you troubleshoot network issues, latency and connectivity problems.
- Useful for monitoring Kubernetes services, ingress, service meshes and CNI plugins.
- Helps you detect security threats such as suspicious network activity or container escapes.
- Works without changing your applications or Kubernetes configuration.

## Key Areas Where eBPF Is Used in Kubernetes

### 1. Kubernetes Networking



- Observe pod to pod traffic.
- Monitor service and ingress traffic.
- See DNS queries and connection details.
- Troubleshoot network policies and CNI issues.

### 2. Observability



- Collect metrics, traces and logs from the kernel.
- See application and system level behaviour.
- Identify performance bottlenecks in real time.
- Works across all pods and nodes without code changes.

### 3. Security



- Monitor system calls and container activity.
- Detect unusual or malicious behaviour.
- Identify privilege escalation or container escape attempts.
- Help enforce security policies and compliance.

### 4. Troubleshooting



- Find the source of network latency.
- See which pod or service is causing issues.
- Trace requests across multiple services.
- Get end to end visibility from the application to the kernel.

### Example 1: Trace Pod to Pod Network Traffic

#### bpftool Program

```
1 tracepoint:tcp:tcp_connect
2 {
3     printf("PID=%d %s:%d -> %s:%d\n",
4         pid, str(args->saddr), args->sport,
5         str(args->daddr), args->dport);
6 }
```

#### What It Shows

- Shows when a pod or process establishes a TCP connection.
- Displays source and destination IP address and port.
- Helps you see pod to pod connections and external calls.

#### Example Output

```
PID=1234 10.244.1.5:54321 -> 10.96.0.10:80
PID=5678 10.244.2.8:40022 -> 10.244.1.7:8080
PID=4321 10.244.1.5:35112 -> 8.8.8.8:53
PID=6789 10.244.3.9:62000 -> 10.96.0.1:443
```

### Example 2: Trace DNS Queries from Pods

#### bpftool Program

```
1 tracepoint:udp:udp_sendmsg
2 /dport == 53/
3 {
4     printf("PID=%d DNS Query: %s\n",
5         pid, str(args->msg));
6 }
```

#### What It Shows

- Shows DNS queries made from pods.
- Displays the process name, PID and query details.
- Helps you identify which application is making DNS requests.

#### Example Output

```
PID=1111 DNS Query: api.kubernetes.default
PID=2222 DNS Query: registry.k8s.io
PID=3333 DNS Query: myservice.default.svc
PID=4444 DNS Query: external-api.com
```

### Example 3: Trace Kubernetes Pod System Calls

#### bpftool Program

```
1 tracepoint:syscalls:sys_enter_openat
2 {
3     printf("PID=%d COMM=%s FILE=%s\n",
4         pid, comm, str(args->filename));
5 }
```

#### What It Shows

- Shows file access inside pods.
- Displays the process name, PID and file path.
- Helps you understand what files applications are reading or writing.

#### Example Output

```
PID=1234 COMM=nginx FILE=/etc/nginx/conf.d/app.conf
PID=2345 COMM=python FILE=/var/log/app.log
PID=6789 COMM=java FILE=/app/config/application.yml
PID=4321 COMM=myapp FILE=/var/lib/data/database.db
```



### Junior Engineer Tip

- Use eBPF tools to observe your cluster first.
- Start with network traffic and DNS queries.
- Always verify findings with application logs and Kubernetes events.



### Real World Use Cases

- Troubleshoot pod to pod communication issues.
- Identify performance bottlenecks in microservices.
- Detect suspicious or malicious container activity.
- Monitor service mesh and ingress traffic.
- Improve security and compliance in Kubernetes.



### Remember This

- eBPF gives visibility, it does not change your app.
- Always consider the full context before taking action.
- Use eBPF together with logs, metrics and traces.
- Be careful with sensitive data in production.
- Understand your cluster networking and CNI.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 23 Kubernetes Networking with Cilium

Pods → eBPF datapath → network policy → services/network communication

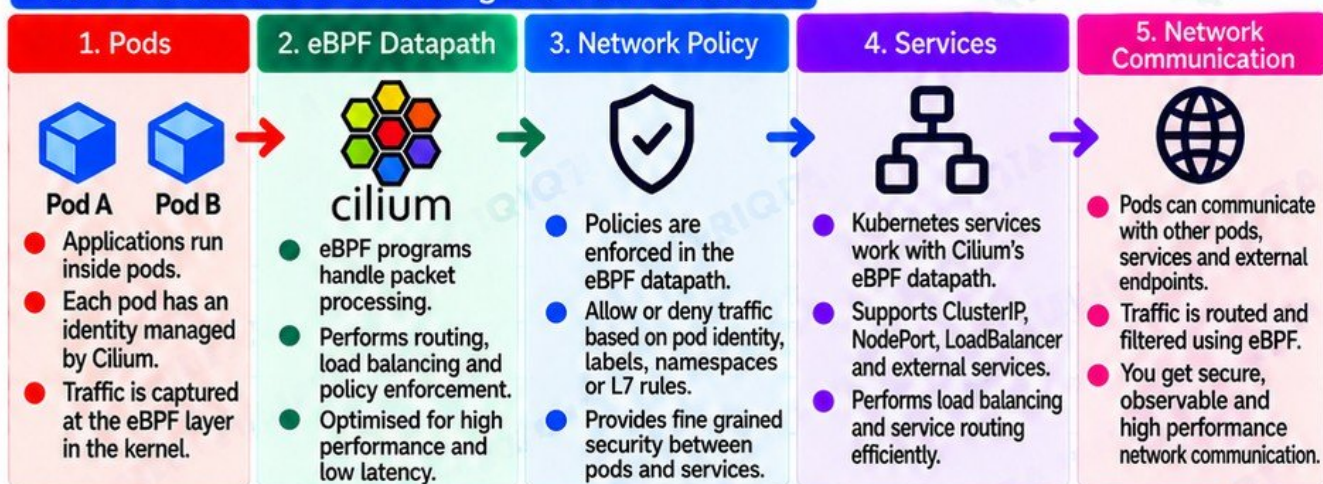
## What Does This Mean?

- Cilium uses eBPF in the Linux kernel to handle Kubernetes networking.
- Traffic between pods, services and the outside world is processed using the eBPF datapath instead of traditional iptables.
- Network policies are enforced at the eBPF layer, giving better performance and visibility.
- This allows secure, fast and observable network communication between pods, services and external endpoints.

## Why Use eBPF for Kubernetes Networking?

- High performance networking with eBPF datapath.
- Advanced and simple to use network policies.
- Deep visibility into network traffic with Hubble (observability).
- Supports Kubernetes services, ingress, egress and multi cluster networking.
- Stronger security with identity based policies instead of only IP addresses.
- Works without modifying your applications.

## How Kubernetes Networking Works with Cilium



## Example 1: Deploy Cilium with eBPF Datapath

### Install Cilium

```
1 helm repo add cilium https://helm.cilium.io
2 helm repo update
3 helm install cilium cilium/cilium \
4   --namespace kube-system \
5   --set k8sServiceHost=${K8S_API_IP} \
6   --set k8sServicePort=6443
```

### What It Shows

- Deploys Cilium in your cluster.
- Uses eBPF for networking, policy and observability.
- Replaces traditional kube-proxy iptables.

### Verify Cilium

```
kubectl get pods -n kube-system
NAME          READY   STATUS    RESTARTS
cilium-abc123 1/1     Running   0
cilium-xyz789 1/1     Running   0
hubble-relay-abc 1/1     Running   0
hubble-ui-xyz 1/1     Running   0
```

## Example 2: Create a Network Policy with Cilium

### Allow traffic only from frontend to backend

```
1 apiVersion: cilium.io/v2
2 kind: CiliumNetworkPolicy
3 metadata:
4   name: allow-frontend-to-backend
5 spec:
6   endpointSelector:
7     matchLabels:
8       app: backend
9   ingress:
10    - fromEndpoints:
11      - matchLabels:
12        app: frontend
13    toPorts:
14      - ports:
15        - port: "80"
16        protocol: TCP
```

### What It Shows

- Creates a network policy using Cilium.
- Allows traffic only from pods with label app=frontend to pods with label app=backend on port 80.
- Enforced at the eBPF layer for high performance.

### Test the Policy

```
# From frontend pod (should work)
kubectl exec -it frontend -- curl http://backend

# From other pod (should be blocked)
kubectl exec -it other -- curl http://backend

# Check policy status
cilium policy get
ENDPOINT  POLICY  IDENTITY  DIRECTION
frontend  allow   1234      ingress
backend   allow   5678      ingress
```

### Junior Engineer Tip

- Start by deploying Cilium and exploring Hubble.
- Use labels to create simple network policies.
- Test connectivity between pods to understand how policies are enforced.
- Always verify the policy behaviour from both allowed and denied sources.

### Real World Use Cases

- ✓ Secure microservices communication in Kubernetes.
- ✓ Observe and troubleshoot network issues with Hubble.
- ✓ Apply zero trust network policies between workloads.
- ✓ Reduce latency with eBPF based networking.
- ✓ Monitor service to service and ingress/egress traffic.

### Remember This

- Cilium enforces network policies at the eBPF layer.
- Pod identity is based on labels, not just IP addresses.
- Always test policies to confirm the expected behaviour.
- Use Hubble for real time network visibility.
- Cilium works with services, ingress, egress and external endpoints.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 24 Observability with Hubble

Understanding Kubernetes network flows and answering "who is talking to whom?"

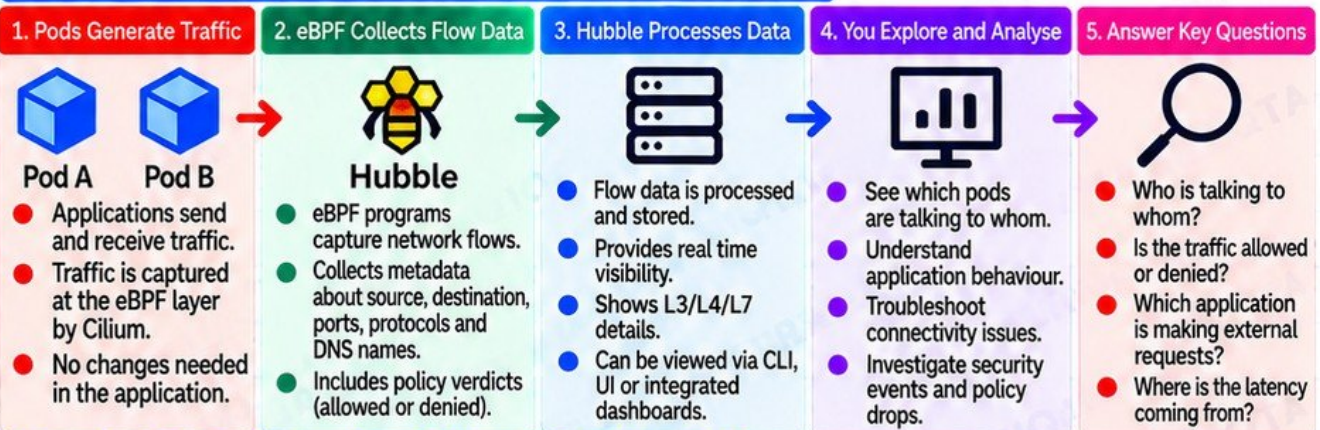
## What Does This Mean?

- Hubble provides deep visibility into Kubernetes network traffic using eBPF.
- It shows which pods, services and external endpoints are communicating.
- You can see real time flows, L3/L4/L7 information and policy decisions.
- It helps you troubleshoot connectivity issues and understand application behaviour.
- This answers the question "who is talking to whom?" inside and outside the cluster.

## Why Use Hubble for Observability?

- Gives real time visibility into Kubernetes network flows.
- Shows source and destination, ports, protocols and DNS names.
- Helps you troubleshoot connectivity, latency and policy issues.
- Shows whether traffic is allowed or denied by Cilium network policies.
- Useful for security monitoring and incident investigation.
- Works without modifying your applications.

## How Hubble Observes Kubernetes Network Flows



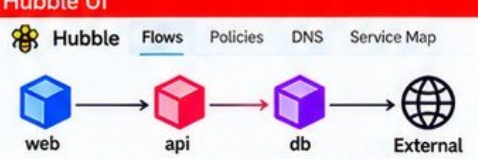
### Example 1: See Real Time Network Flows with Hubble CLI

| Hubble Command                                                                                                                                         | What It Shows                                                                                                                                                                                                                                                                     | Example Output                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |           |        |             |         |    |          |             |             |         |      |          |             |          |         |     |          |             |         |         |       |          |             |            |        |   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|--------|-------------|---------|----|----------|-------------|-------------|---------|------|----------|-------------|----------|---------|-----|----------|-------------|---------|---------|-------|----------|-------------|------------|--------|---|
| <pre>1 hubble observe 2 3 # Show flows in a specific namespace 4 hubble observe -n default 5 6 # Follow flows in real time 7 hubble observe -f 8</pre> | <ul style="list-style-type: none"><li>● Shows real time network flows between pods, services and external endpoints.</li><li>● Displays source, destination, ports, protocol and verdict (allowed or denied).</li><li>● Helps you see actual application communication.</li></ul> | <table><tr><th>TIMESTAMP</th><th>SOURCE</th><th>DESTINATION</th><th>VERDICT</th><th>L7</th></tr><tr><td>10:01:12</td><td>pod/web-abc</td><td>pod/api-xyz</td><td>ALLOWED</td><td>HTTP</td></tr><tr><td>10:01:15</td><td>pod/api-xyz</td><td>kube-dns</td><td>ALLOWED</td><td>DNS</td></tr><tr><td>10:01:18</td><td>pod/web-abc</td><td>1.1.1.1</td><td>ALLOWED</td><td>HTTPS</td></tr><tr><td>10:01:21</td><td>pod/api-xyz</td><td>pod/db-123</td><td>DENIED</td><td>-</td></tr></table> | TIMESTAMP | SOURCE | DESTINATION | VERDICT | L7 | 10:01:12 | pod/web-abc | pod/api-xyz | ALLOWED | HTTP | 10:01:15 | pod/api-xyz | kube-dns | ALLOWED | DNS | 10:01:18 | pod/web-abc | 1.1.1.1 | ALLOWED | HTTPS | 10:01:21 | pod/api-xyz | pod/db-123 | DENIED | - |
| TIMESTAMP                                                                                                                                              | SOURCE                                                                                                                                                                                                                                                                            | DESTINATION                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | VERDICT   | L7     |             |         |    |          |             |             |         |      |          |             |          |         |     |          |             |         |         |       |          |             |            |        |   |
| 10:01:12                                                                                                                                               | pod/web-abc                                                                                                                                                                                                                                                                       | pod/api-xyz                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | ALLOWED   | HTTP   |             |         |    |          |             |             |         |      |          |             |          |         |     |          |             |         |         |       |          |             |            |        |   |
| 10:01:15                                                                                                                                               | pod/api-xyz                                                                                                                                                                                                                                                                       | kube-dns                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | ALLOWED   | DNS    |             |         |    |          |             |             |         |      |          |             |          |         |     |          |             |         |         |       |          |             |            |        |   |
| 10:01:18                                                                                                                                               | pod/web-abc                                                                                                                                                                                                                                                                       | 1.1.1.1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | ALLOWED   | HTTPS  |             |         |    |          |             |             |         |      |          |             |          |         |     |          |             |         |         |       |          |             |            |        |   |
| 10:01:21                                                                                                                                               | pod/api-xyz                                                                                                                                                                                                                                                                       | pod/db-123                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | DENIED    | -      |             |         |    |          |             |             |         |      |          |             |          |         |     |          |             |         |         |       |          |             |            |        |   |

### Example 2: Analyse Specific Traffic (Port, Pod or Protocol)

| Hubble Command                                                                                                                                                                                | What It Shows                                                                                                                                                                                                                                  | Example Output                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |           |        |             |      |       |         |          |         |        |      |     |         |          |         |        |      |     |         |          |         |        |      |     |        |          |         |        |      |     |         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|--------|-------------|------|-------|---------|----------|---------|--------|------|-----|---------|----------|---------|--------|------|-----|---------|----------|---------|--------|------|-----|--------|----------|---------|--------|------|-----|---------|
| <pre>1 # Filter by pod 2 hubble observe --from-pod web-abc 3 4 # Filter by destination port 5 hubble observe --to-port 5432 6 7 # Filter by protocol 8 hubble observe --protocol http 9</pre> | <ul style="list-style-type: none"><li>● Shows flows for a specific pod, port or protocol.</li><li>● Helps you focus on the traffic you are investigating.</li><li>● Useful for troubleshooting application or database connectivity.</li></ul> | <table><tr><th>TIMESTAMP</th><th>SOURCE</th><th>DESTINATION</th><th>PORT</th><th>PROTO</th><th>VERDICT</th></tr><tr><td>10:02:10</td><td>web-abc</td><td>db-123</td><td>5432</td><td>TCP</td><td>ALLOWED</td></tr><tr><td>10:02:11</td><td>web-abc</td><td>db-123</td><td>5432</td><td>TCP</td><td>ALLOWED</td></tr><tr><td>10:02:12</td><td>web-abc</td><td>db-123</td><td>5432</td><td>TCP</td><td>DENIED</td></tr><tr><td>10:02:13</td><td>api-xyz</td><td>db-123</td><td>5432</td><td>TCP</td><td>ALLOWED</td></tr></table> | TIMESTAMP | SOURCE | DESTINATION | PORT | PROTO | VERDICT | 10:02:10 | web-abc | db-123 | 5432 | TCP | ALLOWED | 10:02:11 | web-abc | db-123 | 5432 | TCP | ALLOWED | 10:02:12 | web-abc | db-123 | 5432 | TCP | DENIED | 10:02:13 | api-xyz | db-123 | 5432 | TCP | ALLOWED |
| TIMESTAMP                                                                                                                                                                                     | SOURCE                                                                                                                                                                                                                                         | DESTINATION                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | PORT      | PROTO  | VERDICT     |      |       |         |          |         |        |      |     |         |          |         |        |      |     |         |          |         |        |      |     |        |          |         |        |      |     |         |
| 10:02:10                                                                                                                                                                                      | web-abc                                                                                                                                                                                                                                        | db-123                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 5432      | TCP    | ALLOWED     |      |       |         |          |         |        |      |     |         |          |         |        |      |     |         |          |         |        |      |     |        |          |         |        |      |     |         |
| 10:02:11                                                                                                                                                                                      | web-abc                                                                                                                                                                                                                                        | db-123                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 5432      | TCP    | ALLOWED     |      |       |         |          |         |        |      |     |         |          |         |        |      |     |         |          |         |        |      |     |        |          |         |        |      |     |         |
| 10:02:12                                                                                                                                                                                      | web-abc                                                                                                                                                                                                                                        | db-123                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 5432      | TCP    | DENIED      |      |       |         |          |         |        |      |     |         |          |         |        |      |     |         |          |         |        |      |     |        |          |         |        |      |     |         |
| 10:02:13                                                                                                                                                                                      | api-xyz                                                                                                                                                                                                                                        | db-123                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 5432      | TCP    | ALLOWED     |      |       |         |          |         |        |      |     |         |          |         |        |      |     |         |          |         |        |      |     |        |          |         |        |      |     |         |

### Example 3: Use Hubble UI for Visual Observability

| Hubble UI                                                                           | What It Provides                                                                                                                                                                                                                                               | Key Benefits                                                                                                                                                                                                                                                                              |
|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <ul style="list-style-type: none"> <li>● Visual view of network flows between pods, services and external endpoints.</li> <li>● Shows policy verdicts and protocol details.</li> <li>● Helps you quickly identify unexpected or suspicious traffic.</li> </ul> | <ul style="list-style-type: none"> <li>● Understand application communication easily.</li> <li>● Troubleshoot network issues faster.</li> <li>● Identify security threats and policy drops.</li> <li>● Works with Kubernetes services, ingress, egress and external endpoints.</li> </ul> |

| Junior Engineer Tip                                                                                                                                                                                                    | Real World Use Cases                                                                                                                                                                                                                                                        | Remember This                                                                                                                                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>● Start with Hubble CLI to see basic flows.</li> <li>● Use filters to focus on specific pods or ports.</li> <li>● Use Hubble UI to get a visual view of the network.</li> </ul> | <ul style="list-style-type: none"> <li>● Troubleshoot pod to pod connectivity issues.</li> <li>● Investigate unexpected external traffic.</li> <li>● Verify network policies are working as expected.</li> <li>● Monitor service communication and dependencies.</li> </ul> | <ul style="list-style-type: none"> <li>● Hubble shows real traffic, not just configuration.</li> <li>● Always check policy verdicts (allowed or denied).</li> <li>● Use both CLI and UI for complete visibility.</li> <li>● Hubble requires Cilium to be installed in your cluster.</li> </ul> |

# 25 Runtime Security with eBPF

Observing suspicious process, file, syscall and network behavior

## What Does This Mean?

- eBPF can monitor system activity in real time without changing your applications.
- It allows you to observe process, file, syscall and network behaviour at the kernel level.
- You can detect suspicious activity such as unexpected processes, file access, privilege escalation or unusual network connections.
- This gives you real time visibility and helps you identify threats and investigate incidents faster.

## Why Use eBPF for Runtime Security?

- Provides real time visibility into system activity.
- Helps you detect suspicious behaviour early.
- Monitors process, file, syscall and network events at the kernel level.
- Useful for detecting malware, container escapes, privilege escalation and data exfiltration.
- Works without installing agents inside containers or modifying your applications.
- Lightweight and high performance.

## Key Areas eBPF Monitors for Security

### 1. Process Activity



- Monitor process creation and execution.
- Detect unexpected processes or shells.
- Identify privilege escalation attempts.
- Track parent and child process relationships.

### 2. File System Activity



- Monitor file open, read, write and delete.
- Detect access to sensitive files.
- Identify unexpected file modifications.
- Look for signs of malware or data exfiltration.

### 3. System Calls (Syscalls)



- Observe system calls used by processes.
- Detect unusual or suspicious syscall patterns.
- Identify attempts to bypass security controls.
- Monitor high risk syscalls such as execve, ptrace or mount.

### 4. Network Activity



- Monitor outbound and inbound connections.
- Detect suspicious destinations or ports.
- Identify unusual network patterns or data exfiltration.
- Track DNS queries and unexpected connections from containers or hosts.

## Example 1: Monitor Process Execution with eBPF

### bcc Program (execsnoop)

```
1 # Show new process executions
2 sudo execsnoop
3 # Output
4 TIME PID PPID COMMAND
5 10:01:12 1234 1 /bin/bash
6 10:01:15 5678 1234 /usr/bin/curl
```

### What It Shows

- Displays real time process executions.
- Shows PID, parent PID and command.
- Helps you identify unexpected processes or shells.
- Useful for detecting suspicious activity or malware.

### Example Output

| TIME     | PID  | PPID | COMMAND           |
|----------|------|------|-------------------|
| 10:01:12 | 1234 | 1    | /bin/bash         |
| 10:01:15 | 5678 | 1234 | /usr/bin/curl     |
| 10:01:19 | 9012 | 5678 | /tmp/malicious.sh |

## Example 2: Monitor File Access with eBPF

### bcc Program (opensnoop)

```
1 # Show file open events
2 sudo opensnoop
3 # Output
4 TIME PID COMM FD FILE
5 10:02:11 2345 nginx 3 /etc/nginx/nginx.conf
6 10:02:15 6789 python 4 /var/log/app.log
```

### What It Shows

- Displays real time file open operations.
- Shows which process accessed which file.
- Helps you detect access to sensitive files.
- Useful for incident investigation.

### Example Output

| TIME     | PID  | COMM   | FD | FILE                  |
|----------|------|--------|----|-----------------------|
| 10:02:11 | 2345 | nginx  | 3  | /etc/nginx/nginx.conf |
| 10:02:15 | 6789 | python | 4  | /var/log/app.log      |

## Example 3: Monitor Network Connections with eBPF

### bcc Program (tcpdump)

```
1 # Show top TCP connections
2 sudo tcpdump
3 # Output
4 PID COMM LADDR:PORT RADDR:PORT TX_KB RX_KB
5 1234 curl 10.0.0.10:52344 104.18.22.33:443 12 45
6 5678 python 10.0.0.11:43210 8.8.8.8:53 1 3
```

### What It Shows

- Detects malware TCP connections.
- Shows source, destination and process information.
- Helps you identify suspicious outbound connections.
- Useful for detecting data exfiltration or command and control.

### Example Output

| PID  | COMM   | LADDR:PORT      | RADDR:PORT       | TX_KB | RX_KB |
|------|--------|-----------------|------------------|-------|-------|
| 1234 | curl   | 10.0.0.10:52344 | 104.18.22.33:443 | 12    | 45    |
| 5678 | python | 10.0.0.11:43210 | 8.8.8.8:53       | 1     | 3     |

## Junior Engineer Tip

- Start by monitoring process executions and network connections.
- Look for unexpected processes or outbound traffic.
- Use eBPF tools to get real evidence, not assumptions.

## Real World Use Cases

- Detect malware or malicious processes.
- Identify container escape attempts.
- Monitor access to sensitive files.
- Investigate suspicious network connections.
- Improve security monitoring and incident response.

## Remember This

- eBPF gives visibility, but always investigate further.
- Look for context and verify if the activity is expected.
- Monitor multiple signals (process, file, syscall, network).
- Use eBPF together with alerts, logs and policies.
- Visibility helps you detect and respond faster.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 26 CO-RE and BTF

Why portable eBPF programs need kernel type information and Compile Once – Run Everywhere

## What Does This Mean?

- CO-RE (Compile Once – Run Everywhere) allows eBPF programs to run on different Linux kernel versions without recompiling.
- BTF (BPF Type Format) provides type information about kernel structures, functions and variables.
- CO-RE uses BTF to automatically adjust to differences in kernel layouts, field offsets and structure changes.
- This makes eBPF programs portable, reliable and easier to maintain across multiple kernel versions.
- You can write, compile once, and run the same eBPF program on many kernels.

## Why Use CO-RE and BTF?

- No need to recompile for different kernel versions.
- Automatically adapts to kernel structure changes.
- Reduces maintenance and complexity.
- Makes eBPF tools portable across distributions and environments.
- Enables powerful observability, security and networking tools to work everywhere.
- Helps you focus on writing logic instead of handling kernel version differences.
- Widely used by modern eBPF tools like Cilium, Falco, Tetragon, Pixie and many others.

## How CO-RE and BTF Work Together

### 1. Kernel Provides BTF



- The kernel exposes type information using BTF.
- BTF contains details about kernel structures, functions and variables.
- This information describes the actual kernel layout.

### 2. Compile Once with CO-RE



- You write your eBPF program using CO-RE helpers and compile it once.
- The compiled program contains relocation information instead of hardcoded kernel offsets.
- It does not depend on a specific kernel version.

### 3. Load on Any Kernel



- When the program loads, it reads the kernel's BTF information.
- CO-RE automatically adjusts structure offsets, field names and layout differences.
- The same compiled program works on different kernels.

### 4. Run Everywhere



- The eBPF program runs correctly on multiple kernel versions.
- No manual changes or recompilation needed.
- You get a single portable binary for observability, security or networking use cases.

## Example 1: View BTF Information

### Command

```
# Check if kernel has BTF enabled
ls /sys/kernel/btf

# List available BTF files
ls /sys/kernel/btf/
```

### What It Shows

- Confirms BTF is available on your system.
- Shows the BTF file for the running kernel.
- This contains type information used by CO-RE programs.

### Example Output

```
/sys/kernel/btf/vmlinux
/sys/kernel/btf/modules

vmlinux      # main kernel BTF
nf_conntrack # example module BTF
tcp_bpf      # example module BTF
```

## Example 2: Compile an eBPF Program with CO-RE

### Command

```
1 #Compile eBPF program with CO-RE
2 clang -O2 -g -target bpf \
4 -D__TARGET_ARCH_x86 \
5 -c myprogram.bpf.c -o myprogram.bpf.o
```

### What It Shows

- Compiles the eBPF program once using CO-RE.
- The program contains relocation information instead of fixed offsets.
- The same binary can run on different kernel versions.

### Example Output

```
myprogram.bpf.o # portable eBPF binary
                 (no kernel specific offsets)
```

## Example 3: Run the eBPF Program

### Command

```
1 # Load using bpftool
2 sudo bpftool prog load myprogram.bpf.o \
4 /sys/fs/bpf/myprog
```

### What It Shows

- Loads the compiled CO-RE program into the kernel.
- The kernel uses its BTF information to resolve correct structure offsets.
- The program runs without recompilation.

### Example Output

```
124: bpf_prog_load: success
loaded program id 124
```



### Junior Engineer Tip

- Always check if your kernel has BTF enabled.
- Use CO-RE to write portable eBPF programs.
- You do not need to know exact kernel offsets.
- Focus on logic, CO-RE handles the differences.



### Real World Use Cases

- ✓ Observability tools like Cilium and Pixie.
- ✓ Security tools like Falco and Tetragon.
- ✓ Network monitoring and troubleshooting.
- ✓ Works across different Linux distributions and kernel versions.



### Remember This

- BTF provides kernel type information.
- CO-RE makes eBPF programs portable.
- Compile once and run everywhere.
- No manual kernel version handling required.
- Enabled by modern eBPF toolchains.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 27 eBPF Performance & Overhead

Why observability itself has a cost and how engineers think about safe instrumentation

## What Does This Mean?

- eBPF programs run in the Linux kernel, so they consume CPU, memory and other kernel resources.
- Observability tools built with eBPF (such as Cilium, Hubble, Pixie, Falco, Tetragon) add overhead because they collect, process and export data.
- The overhead is usually low when designed correctly, but it can become significant if you capture too many events or use expensive operations.
- You need to understand and measure the cost so that observability remains safe in production.

## Why Performance and Overhead Matter?

- Helps you make informed decisions about what to collect and what to avoid.
- Prevents unnecessary load on the kernel and your applications.
- Ensures observability does not impact the performance or stability of your workloads.
- Helps you design efficient and targeted instrumentation.
- Important for production environments where resources are shared and latency matters.
- Allows you to balance visibility, cost and value.

## Factors That Affect eBPF Overhead

### 1. Event Volume



- The more events you capture, the higher the overhead.
- High frequency events such as syscalls or network packets can generate a lot of data.
- Use filters to capture only what you need.

### 2. Program Complexity



- Complex eBPF programs use more CPU cycles in the kernel.
- Expensive operations such as loops, map lookups or string processing can increase overhead.
- Keep programs small, efficient and focused on specific tasks.

### 3. Data Export and Processing



- Sending data to user space, storing it or exporting to external systems adds overhead.
- High volume of logs, metrics or traces can consume CPU, memory and network bandwidth.
- Use batching, sampling and filtering to reduce the amount of data.

### 4. Resource Usage



- eBPF uses kernel CPU, memory and map storage.
- Large maps or high cardinality data can increase memory usage.
- Always monitor resource usage when running eBPF based tools.

## Example 1: Measure eBPF CPU Usage

### Command

```
# Show eBPF program CPU usage
sudo bpftool prog profile \
  id <PROGRAM_ID> duration 10
```

### What It Shows

- Profiles the CPU usage of an eBPF program.
- Helps you understand how much CPU time the program is using.
- Useful for comparing performance after making changes.

### Example Output

```
Duration: 10.000s
CPU time: 12.345ms
Processed events: 1,234,567
Average per event: 10ns
```

## Example 2: Check eBPF Map Memory Usage

### Command

```
# Show memory used by eBPF maps
sudo bpftool map show
```

### What It Shows

- Displays all eBPF maps and their memory usage.
- Helps you identify large maps or high cardinality data.
- Useful for optimisation and capacity planning.

### Example Output

| ID | TYPE     | NAME        | MEM_USED |
|----|----------|-------------|----------|
| 12 | hash     | events_map  | 8.0M     |
| 13 | lru_hash | connections | 16.0M    |
| 14 | array    | metrics     | 1.0M     |

## Example 3: Use Filters to Reduce Overhead

### Command

```
# Trace only execve syscalls instead
# of all syscalls
sudo tracee-ebpf --events execve
```

### What It Shows

- Captures only specific events instead of everything.
- Reduces the amount of data collected.
- Helps you focus on the information that matters.

### Example Output

| TIME     | PID  | COMM   | EVENT  |
|----------|------|--------|--------|
| 10:01:12 | 1234 | bash   | execve |
| 10:01:15 | 5678 | python | execve |
| 10:01:19 | 9012 | nginx  | execve |



### Junior Engineer Tip

- Start with minimal event collection.
- Use filters, sampling and batching.
- Monitor CPU and memory usage of eBPF tools.
- Always test in a non production environment first.



### Real World Use Cases

- ✓ Monitor the performance impact of eBPF tools.
- ✓ Optimise event collection for production.
- ✓ Investigate high CPU or memory usage.
- ✓ Balance visibility with resource consumption.
- ✓ Ensure observability remains safe and reliable.



### Remember This

- Observability always has a cost.
- Capture only what you need.
- Keep eBPF programs efficient and simple.
- Monitor resource usage regularly.
- Performance and visibility must be balanced.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 28 eBPF Production Safety

Privileges, kernel compatibility, verifier failures, resource limits and operational risk

## What Does This Mean?

- eBPF runs in the Linux kernel, so safety is critical before using it in production.
- You need to understand the required privileges, kernel compatibility, verifier restrictions, resource limits and operational risks.
- Proper safety practices ensure eBPF programs provide real value without affecting system stability or security.
- This helps you run eBPF tools like Cilium, Hubble, Falco, Tetragon and custom programs safely in real environments.

## Why Production Safety Matters?

- Prevents accidental system instability or crashes.
- Reduces security risks from overly permissive access.
- Helps you avoid verifier failures and program rejection.
- Ensures compatibility across different kernel versions and distributions.
- Keeps eBPF programs within resource limits (CPU, memory, maps, programs).
- Minimises operational risk in production environments.

## Key Areas of eBPF Production Safety

### 1. Privileges & Access



- eBPF programs require Linux capabilities.
- Commonly needs CAP\_BPF and CAP\_SYS\_ADMIN.
- In containers, use minimal privileges.
- Avoid running with full root access when not required.

### 2. Kernel Compatibility



- eBPF features depend on kernel version.
- Not all helpers or program types are available on older kernels.
- Check kernel version and supported features before deployment.
- Use CO-RE to improve compatibility across kernel versions.

### 3. Verifier Failures



- eBPF programs are checked by the kernel verifier before running.
- Programs can be rejected due to unsafe memory access, loops or invalid logic.
- Follow verifier friendly coding practices.
- Always check verifier logs when a program fails to load.

### 4. Resource Limits



- eBPF has limits on program size, map size, stack size and number of instructions.
- Large maps can use significant memory.
- High event volumes can consume CPU.
- Monitor resource usage and set appropriate limits.

### 5. Operational Risk



- Poorly written programs can impact system performance.
- High event volume can create overhead.
- Conflicts with other eBPF tools are possible.
- Always test in a staging environment first.
- Have a rollback plan and monitoring in place.

## Example 1: Check Required Capabilities

### Command

```
# Check process capabilities
getpcaps $$

# Check capabilities for a container
docker exec -it <container> getpcaps 1
```

### What It Shows

- Shows which capabilities are available to the process.
- Helps you verify if CAP\_BPF and CAP\_SYS\_ADMIN are set.
- Useful for troubleshooting permission errors.

### Example Output

```
$ getpcaps $$
1234: = cap_bpf,cap_sys_admin,
      cap_net_admin+ep
```

## Example 2: Verifier Failure Message

### Command

```
# Load an eBPF program and check logs
sudo bpftool prog load ./example.o /sys/fs/bpf/
# If it fails, check the kernel log
dmesg | tail -n 50
```

### What It Shows

- Shows why the program was rejected by the verifier.
- Helps you identify the exact line and reason for failure.
- Common reasons include invalid memory access, loops and unsupported helpers.

### Example Output

```
bpf: prog 'example': rejected!
bpf: invalid memory access 'map_value'
bpf: R1 invalid mem access 'map_value'
bpf: verification failed
```

## Example 3: Check Resource Usage

### Command

```
# Show loaded eBPF programs
sudo bpftool prog show

# Show map usage
sudo bpftool map show
```

### What It Shows

- Displays currently loaded programs and maps.
- Shows memory usage, type and number of entries.
- Helps you monitor resource consumption.

### Example Output

```
$ sudo bpftool prog show
123: tracing name xdp_monitor tag a1b2c3d4
$ sudo bpftool map show
34: hash name events_map flags 0x0
   key 4B value 128B max_entries 65536
   memlock 836868B
```

## Junior Engineer Tip

- Always test eBPF programs in a non production environment first.
- Start with existing tools like Cilium or Hubble to learn safe practices.
- Monitor system performance after deployment.

## Real World Use Cases

- Running eBPF based security tools like Falco.
- Network observability with Cilium and Hubble.
- System monitoring and performance analysis.
- Detecting suspicious behaviour in real time.
- Using eBPF in containers and Kubernetes safely.

## Remember This

- Use the minimum required privileges.
- Check kernel compatibility.
- Always review verifier logs.
- Monitor CPU and memory usage.
- Test before production and have a rollback plan.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 29 Production Troubleshooting Workflow

Symptom → evidence → eBPF observation → hypothesis → root cause → fix → verification

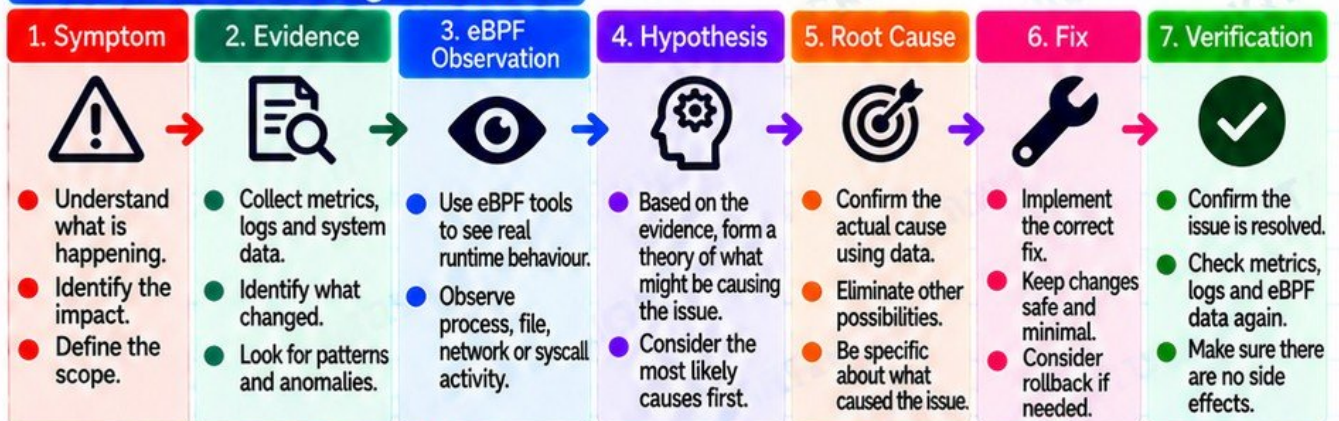
## What Does This Mean?

- A structured way to troubleshoot production issues using real evidence and eBPF data.
- Helps you move from a problem report to the actual root cause instead of guessing.
- Combines system metrics, logs and eBPF observations to understand what is really happening.
- Reduces time to resolution and avoids unnecessary changes.

## Why Use This Workflow?

- Keeps troubleshooting organised and focused.
- Helps you collect the right evidence at each step.
- Uses eBPF to see real runtime behaviour inside the kernel and containers.
- Avoids random fixes that can make the problem worse.
- Works for performance, networking, security and application issues.
- Improves reliability and production stability.

## The Troubleshooting Workflow



### 1. Symptom: Identify the Issue

# Example: High latency in an application  
User reports slow response time  
Check current status  
kubectl get pods  
kubectl top pods

#### What It Shows

- Confirms the issue and its impact.
- Helps you see which components are affected.
- Gives you a starting point for further investigation.

### 4. Hypothesis: Form a Theory

# Based on the evidence  
Possible causes:  
- High CPU usage  
- Slow database queries  
- Network latency to external service

#### What It Shows

- Helps you focus on the most likely causes.
- Prevents random changes.
- Guides which eBPF data to investigate next.

### 7. Verification: Confirm Resolution

# Verify the issue is resolved  
kubectl top pods  
hubble observe --since 5m  
kubectl logs <pod-name> --tail 50

#### What It Shows

- Confirms the issue is fixed.
- Checks that metrics, logs and eBPF data are normal.
- Ensures there are no side effects or new issues.

### 2. Evidence: Collect Data

# Check metrics and logs  
kubectl top pods  
kubectl logs <pod-name>  
kubectl get events --sort-by=.metadata  
.lastTimestamp

#### What It Shows

- Shows resource usage and errors.
- Helps you identify recent changes.
- Provides context before using eBPF for deeper analysis.

### 5. Root Cause: Confirm the Cause

# Example: Find the process causing high I/O  
sudo tracee-eBPF --events write,read  
# Example: Check network connections  
hubble observe --protocol http --port 5432

#### What It Shows

- Confirms the exact component causing the issue.
- Provides real evidence from eBPF data.
- Helps you eliminate other possibilities.

### 3. eBPF Observation

# Example: Observe network activity with Hubble  
hubble observe --pod <pod-name>  
# Observe syscall activity with Tracee  
sudo tracee-eBPF --follow

#### What It Shows

- Shows real time network, process or syscall behaviour.
- Helps you see what the application is actually doing.
- Reveals hidden issues not visible in normal logs.

### 6. Fix: Implement the Solution

# Example: Restart a pod or adjust configuration  
kubectl rollout restart deployment <name>  
# Or apply a config change  
kubectl apply -f new-config.yaml

#### What It Shows

- Applies the correct fix based on the root cause.
- Uses minimal and safe changes.
- Keeps a rollback plan if needed.

### Junior Engineer Tip

- Always follow the workflow step by step.
- Use eBPF tools to get real evidence, not guesses.
- Start with read only commands before making changes.
- Compare before and after data.
- Document what you found and what you changed.
- If the fix does not work, go back to the evidence and recheck your hypothesis.

### Remember This

- Collect evidence before making changes.
- Use eBPF to see real runtime behaviour.
- Do not assume the first thing you see is the root cause.
- Keep changes small and reversible.
- Always verify that the issue is fully resolved.
- Good troubleshooting improves reliability and builds trust in production.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy

# 30 Production Incident Challenge

A realistic incident combining Kubernetes, networking, application latency and eBPF evidence

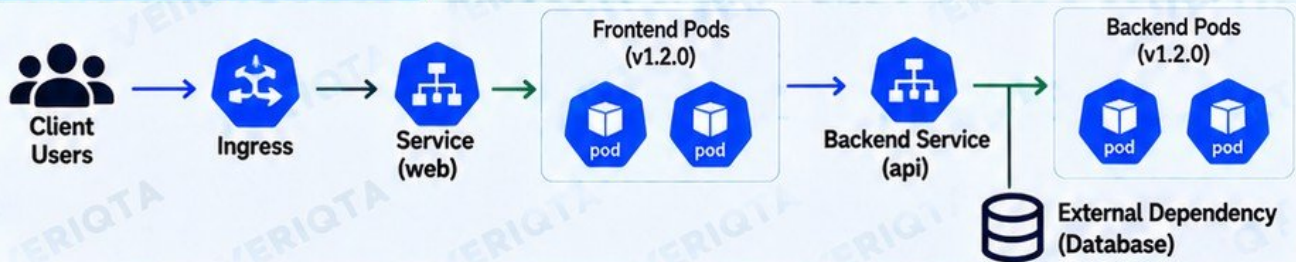
## Incident Scenario

- A production e-commerce application is experiencing high latency and intermittent timeouts.
- The issue started 20 minutes ago after a new version was deployed.
- Some users can access the application, but many requests are slow.
- Infrastructure metrics show that CPU and memory are normal.
- You need to investigate using Kubernetes tools, network information and eBPF evidence to find the real cause and fix it.

## Key Questions to Answer

- What is causing the high latency?
- Is the problem in the application, networking or a dependency?
- Which pods, services or endpoints are affected?
- Are there any network drops, retransmissions or DNS issues?
- What does the eBPF data show about the traffic?
- How do you confirm the root cause and fix it?
- What steps will you take to prevent this in the future?

## Environment Overview



## Symptoms Observed

- Users report slow response time.
- Some requests time out.
- Frontend pods show high request duration.
- Backend service appears healthy at first glance.
- CPU and memory on nodes are normal.

## Key Evidence to Investigate

- Kubernetes pod, service and endpoint status.
- Application logs and request latency.
- Network latency, drops or retransmissions.
- DNS resolution and external dependency calls.
- eBPF data for real network and syscall visibility.
- Compare behaviour before and after the deployment.

## Investigation Flow

- Check application and Kubernetes status.
- Collect network and DNS information.
- Use eBPF tools to observe live traffic and system behaviour.
- Form a hypothesis from the evidence.
- Identify the root cause.
- Implement the fix.
- Verify that the issue is resolved.

### 1. Check Pod and Service Status

```
kubectl get pods -n shop
kubectl get svc -n shop
kubectl get endpoints -n shop
kubectl describe pod <pod-name> -n shop
```

### 2. Check Application Logs

```
kubectl logs <pod-name> -n shop
kubectl logs <pod-name> -n shop \
  --previous
kubectl logs -l app=api -n shop --tail 100
```

### 3. Observe Network with eBPF

```
# Observe HTTP requests and latency
sudo tracee-eBPF -t http
sudo tcpconnect -p
sudo opensnoop-bpfcc
sudo funclatency-bpfcc -i 1 'tcp_sendmsg'
```

### 4. Check DNS and External Dependency

```
kubectl exec -it <pod-name> -n shop -- nslookup
kubectl exec -it <pod-name> -n shop -- curl -v
kubectl exec -it <pod-name> -n shop -- \
  dig database.shop.svc.cluster.local
sudo tracee-eBPF -t dns
```

### 5. Analyse eBPF Output

```
# Look for high latency, retransmissions or drops
TIME SRC DEST LAT(ms) EVENT
10:02:11 10.1.2.5 10.1.3.8 450 http
10:02:12 10.1.2.5 10.1.3.8 520 retry
10:02:13 10.1.2.5 10.1.3.8 600 timeout
```

### 6. Identify and Fix the Root Cause

```
# Example fix (roll back or update config)
kubectl rollout undo deployment/api -n shop
kubectl set env deployment/api -n shop \
  DB_POOL_SIZE=50
kubectl rollout restart deployment/api -n shop
```

## Example Root Cause (One Possible Outcome)

- eBPF shows high latency and retransmissions from the API pods to the database.
- Connection pool was misconfigured in the new release.
- This caused many requests to wait, increasing latency and timeouts.

## Verification Steps

- Check application response time is normal.
- Confirm eBPF no longer shows high latency or retransmissions.
- Verify error rates are back to normal.
- Monitor for at least 15 to 30 minutes.

## Key Takeaways

- Use real evidence, not assumptions.
- eBPF gives deep visibility into what is happening at the network and system level.
- Combine Kubernetes, application and network data to find the real cause.
- Always verify the fix and monitor after changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta\_academy